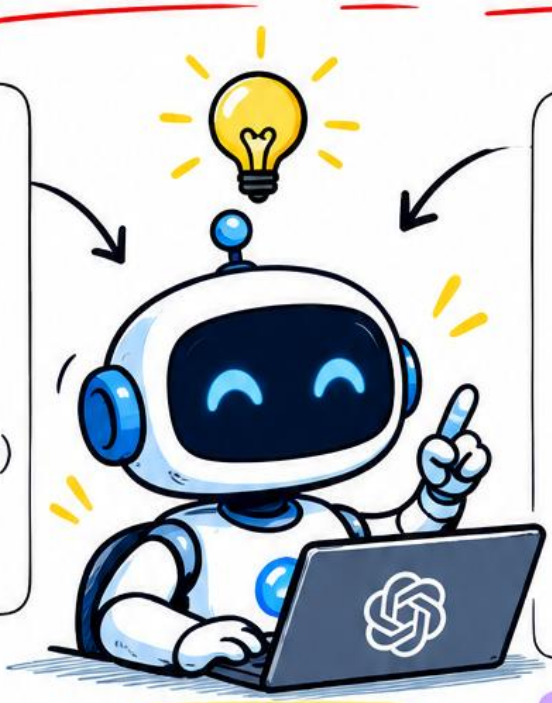
Complete Generative AI

Cheat Sheet + 100 Interview Questions

Key Concepts | Architecture | Tools | Real World Projects

Core Concepts

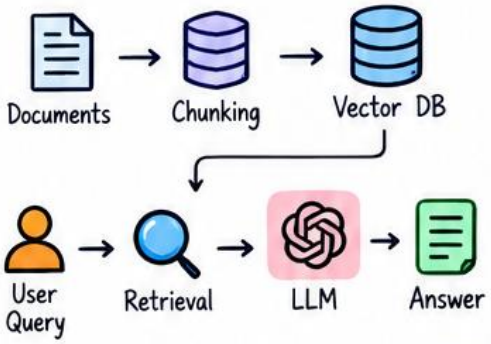
- [x] What is Gen AI?
- [x] How it Works?
- [x] LLMs & Prompting
- [x] Tokenization
- [x] Embeddings
- [x] RAG
- [x] Fine-Tuning (LoRA/PEFT)
- [x] AI Agents
- [x] Multimodal AI



Popular LLMs

- OpenAI — GPT-4o / GPT-4
- Meta — Llama 3
- Google — Gemini
- Anthropic — Claude
- Mistral AI
- DeepSeek

RAG Pipeline



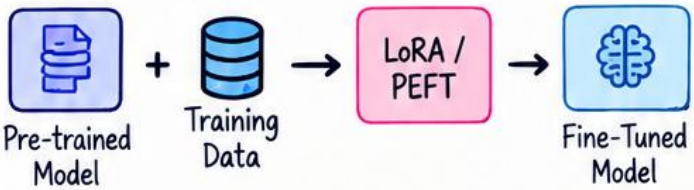
AI Agents



Real World Use Cases

- Chatbot / Assistant
- Content Generation
- Data Analysis
- Productivity & Automation

Fine-Tuning (LoRA / PEFT)



What You Get

- [x] One Page Cheat Sheet
- [x] 100 Interview Questions (with Answers)
- [x] Code Examples (Python)
- [x] Architecture Diagrams
- [x] Best Practices
- [x] Real World Project (End-to-End)

1. Generative AI – Zero to Hero

Complete Notes + Cheat Sheet

From Basics to Real-World AI

- **What You Will Learn ?**

- [x] Core AI Concepts
- [x] LLMs & Transformers
- [x] Prompt Engineering
- [x] RAG + Vector Databases
- [x] AI Agents
- [x] LangChain & LangGraph
- [x] Fine-Tuning
- [x] Real-World Projects
- [x] Interview Preparation

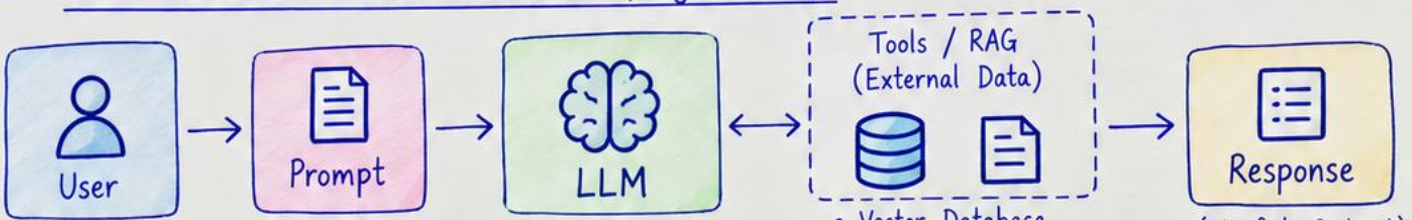


"Turn Ideas into Real-World AI Applications"



Understand
Build
Experiment
Apply
Grow !

- **How Generative AI Works ? (High Level)**



- Vector Database
- APIs
- Real-time Data
- Documents (PDF, URL, etc.)

- **Why Learn Generative AI ?**



Learn • Build • Apply • Grow

2. AI vs ML vs Deep Learning vs Generative AI

Artificial Intelligence (AI)

- Broad field of making machines intelligent like humans.
- Includes problem solving, reasoning, decision making, planning, etc.
- AI can be rule-based (traditional) or can use ML, DL, etc.

Examples :

- Chess Playing
- Robotics
- Expert Systems
- Self-driving Cars

Machine Learning (ML)

- Subset of AI.
- Learns from data without being explicitly programmed.
- Uses statistical techniques and algorithms.
- Finds patterns in data and makes predictions or decisions.

Examples :

- Spam Detection
- Price Prediction
- Customer Segmentation
- Disease Prediction

Deep Learning (DL)

- Subset of ML.
- Uses Artificial Neural Networks (ANN) with multiple layers.
- Learns complex patterns from large amount of data.
- Works very well for images, text, audio, video, etc.

Examples :

- Image Recognition
- Speech Recognition
- Language Translation
- Object Detection (in Videos)

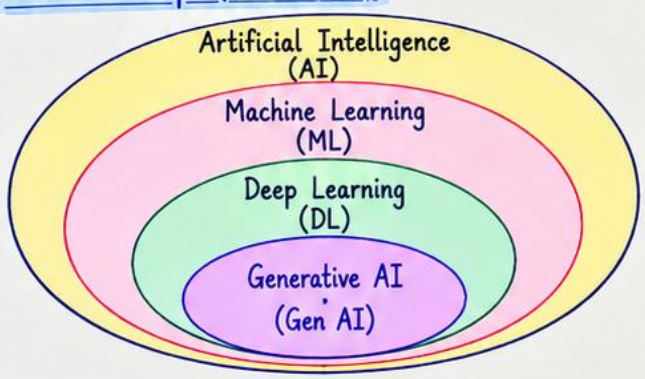
Generative AI (Gen AI)

- Subset of Deep Learning (and ML).
- Generates new content (text, images, audio, video, code, etc.)
- Learns the underlying patterns and creates similar new data.
- Uses models like LLMs, Diffusion Models, GANs, etc.

Examples :

- ChatGPT (Text)
- DALL·E (Images)
- Music Generation
- Code Generation (GitHub Copilot)

Relationship (Set View)



Simple Analogy

AI →	A smart human mind that can think and make decisions.
ML →	A person who learns from experience (data) without being told exact rules.
DL →	A person with a very powerful brain (multiple layers) who can learn complex patterns.
Gen AI →	A creative person who can create new content (text, images, code, etc.) based on what they have learned.

Key Differences

Feature	AI	ML	Deep Learning	Generative AI
Scope	Broadest	Subset of AI	Subset of ML	Subset of DL
Learning	Rule-based or Data-driven	Learns from data	Learns using Neural Networks	Learns & generates new content
Data Need	Low to High	Medium	High	Very High
Model Type	Rules / Algorithms	Statistical Models	Neural Networks	LLMs, Diffusion, GANs, etc.
Output	Decisions / Actions	Predictions	Pattern Recognition	New Content (Text, Image, Code, etc.)

Key Takeaways

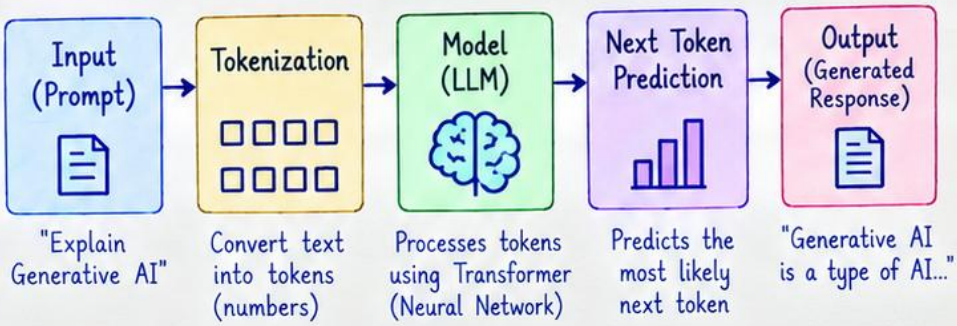
- ☑ AI is the broad field.
- ☑ ML is a subset of AI.
- ☑ Deep Learning is a subset of ML.
- ☑ Generative AI is a subset of Deep Learning.
- ☑ Generative AI can create new content.
- ☑ All Gen AI models are not LLMs (e.g., Diffusion Models for images).

3. How Generative AI Works

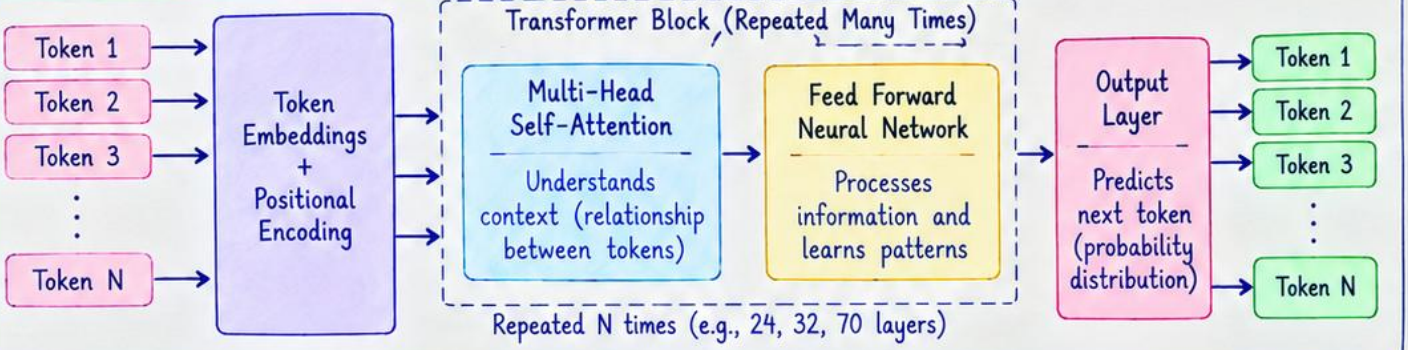
1. Basic Idea

- Generative AI learns patterns from large amount of data (text, images, audio, code, etc.) and generates new content similar to what it has learned.
- It does not just retrieve data, it understands context and predicts the next most likely output.
- Uses advanced models like Transformers and Large Language Models (LLMs).

2. Overall Workflow



3. Inside the Model (Transformer)



4. Tokenization Example

Text: "Generative AI is amazing!"

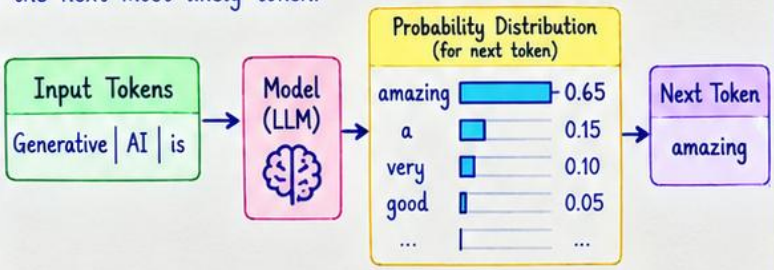
↓ Tokenization (splits text into smaller tokens)

Generative	AI	is	amazing	!
5021	318	374	10432	0

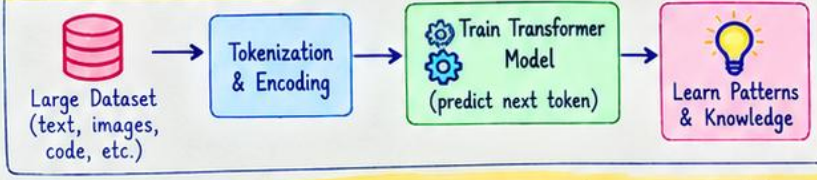
Each token is converted to a unique number (using a tokenizer like BPE - Byte Pair Encoding).

5. Next Token Prediction (Simplified)

The model looks at previous tokens and predicts the next most likely token.



6. Training Process



7. Inference (Real World Use)



Learn → Understand → Build → Apply → Grow

4. Tokens, Embeddings & Vector Representation

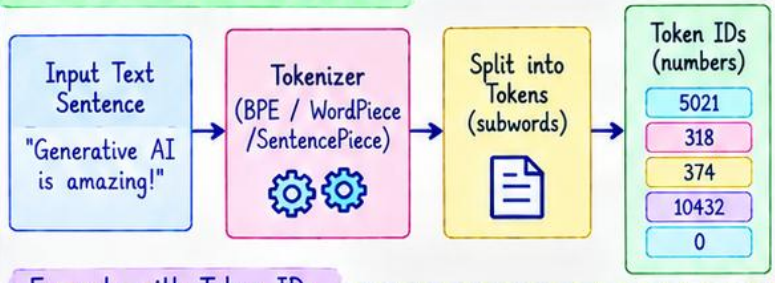
1. What are Tokens ?

- Tokens are small pieces of text that the model reads and processes.
- A token can be a word, part of a word, or even a single character.
- LLMs do not read full sentences directly, they read a sequence of tokens.

Examples :

Sentence	I love Generative AI !
Tokens (approx)	I | love | Gener | ative | AI | !
Number of tokens	6 tokens

2. Tokenization Process



Example with Token IDs

Text:	"Generative AI is amazing!"					
Tokens:	Gener	ative	AI	is	amazing	!
Token IDs:	5021	318	374	10432	845	0

3. What are Embeddings ?

- Embeddings convert tokens (text) into numerical vectors (list of numbers).
- These vectors capture the meaning and context of the text.
- Tokens with similar meaning will have similar vector representations.

Embedding Example (Simplified)

Word / Token	Embedding Vector (5 dimensions)
cat	[0.12, 0.45, -0.21, 0.33, 0.09]
dog	[0.10, 0.42, -0.25, 0.31, 0.11]
car	[-0.30, 0.18, 0.50, -0.12, 0.40]

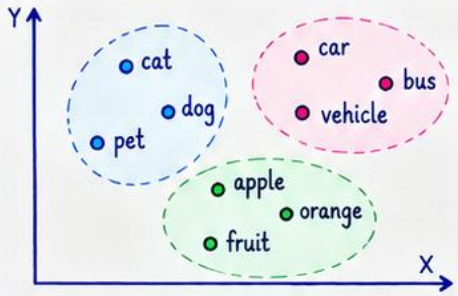
Key Points :

- ☑ Embeddings are dense vectors of floating point numbers.
- ☑ Capture semantic meaning.
- ☑ Similar words have similar embeddings.
- ☑ Used for search, similarity, clustering, recommendation, etc.

4. Vector Representation

- A vector is a list of numbers that represents a token, word, sentence or document in a multi-dimensional space.
- Higher dimensions (e.g., 384, 768, 1536) can capture more meaning.
- These vectors allow the model to understand similarity between text.

2D Vector Space (Simple Visualization)



5. Semantic Similarity

Words with similar meaning have similar vectors even if the words are different.

Text Pair	Similarity Score (cosine similarity)
cat - dog	0.92
car - vehicle	0.89
apple - orange	0.87
cat - car	0.12

6. Embedding Dimensions (Common Models)

Model	Embedding Dimension
OpenAI (text-embedding-3-small)	1536
OpenAI (text-embedding-3-large)	3072
Sentence Transformers (all-MiniLM)	384
BERT (base)	768
LLaMA (sentence embeddings)	4096

7. Applications

- Semantic Search
- Document Similarity
- RAG (Retrieval Augmented Generation)
- Recommendation Systems
- Clustering & Classification
- Chatbots (Context Understanding)

8. Key Takeaways

- ☑ Tokens are small pieces of text.
- ☑ Tokenization converts text to tokens and token IDs.
- ☑ Embeddings convert tokens to numerical vectors.
- ☑ Vectors capture meaning and context.
- ☑ Similar text has similar vectors.
- ☑ Used in search, RAG, chatbots, and many real-world AI applications.

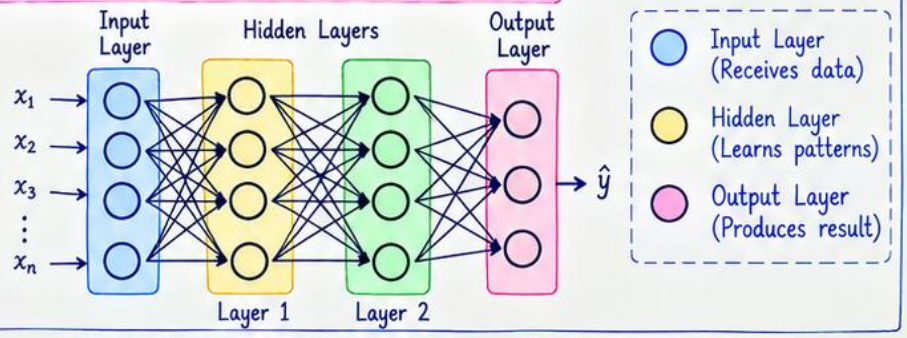
Text → Tokens → Embeddings → Vector Space → Meaning → Real World Applications

5. Neural Networks & Transformers

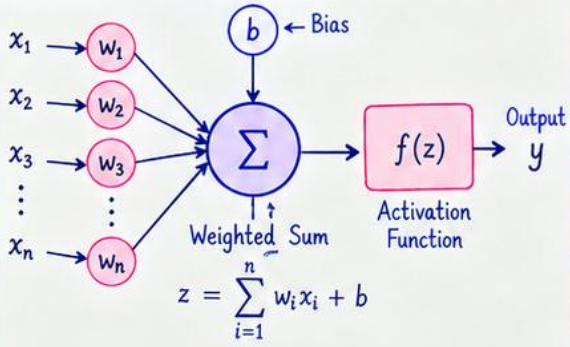
1. What is a Neural Network ?

- A Neural Network is a machine learning model inspired by the human brain.
- It consists of layers of interconnected nodes (neurons) that learn patterns from data.
- Each connection has a weight, and neurons apply an activation function to learn complex relationships.

2. Basic Structure of a Neural Network



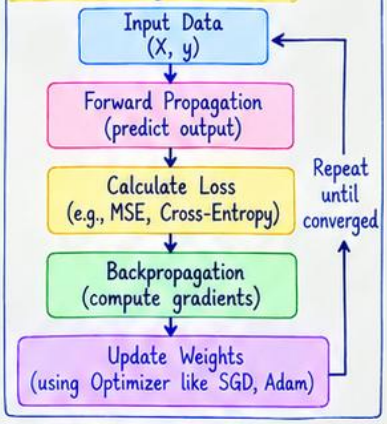
3. Neuron Computation



4. Common Activation Functions

Function	Formula	Graph
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	
ReLU	$f(x) = \max(0, x)$	
Tanh	$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	
Leaky ReLU	$f(x) = \max(\alpha x, x)$	
Softmax	$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$	

5. Training Process

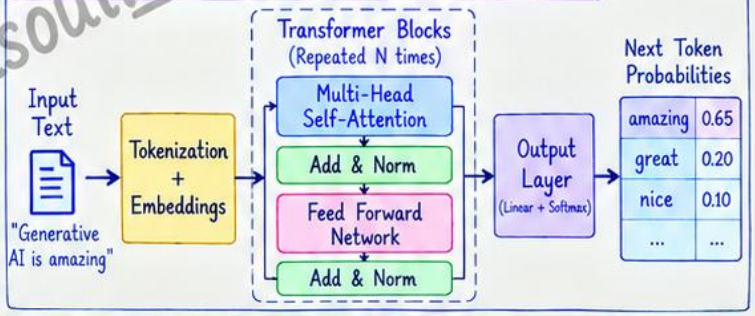


6. What is a Transformer ?

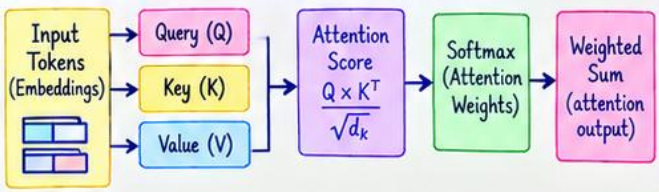
- A Transformer is a deep learning model architecture introduced in the paper "Attention is All You Need" (2017).
- It relies on Self-Attention instead of recurrence (RNN/LSTM).
- It can process all tokens in parallel, making it faster and more effective for large-scale data (like LLMs).
- It is the foundation of modern models like GPT, BERT, T5, LLaMA, etc.

@codedsoul_05

7. Transformer Architecture (High Level)



8. Self-Attention Mechanism (Simplified)



Self-Attention allows the model to focus on relevant tokens in the input sequence, no matter their position.

9. Advantages of Transformers

- ☑ Captures long-range dependencies
- ☑ Processes tokens in parallel
- ☑ More scalable than RNN/LSTM
- ☑ Works well for large datasets
- ☑ Foundation of modern LLMs
- ☑ Effective for text, images, audio, and multimodal data

10. Real World Models Using Transformers

- GPT (Text Generation)
- BERT (Language Understanding)
- LLaMA (Open Source LLM)
- T5 (Text-to-Text)
- Stable Diffusion (Image Generation)
- Whisper (Speech Recognition)

Neural Networks → Transformers → Powerful AI Models

Learn • Build • Apply • Grow

@codedsoul_05

6. Large Language Models (LLMs)

1. What are LLMs ?

- Large Language Models (LLMs) are very large AI models trained on massive amounts of text data.
- They understand, generate, and manipulate human language.
- Built using Transformer architecture.
- Examples: GPT, LLaMA, Claude, Gemini, etc.

2. Key Characteristics

- ☑ Trained on massive text data (internet, books, articles, etc.)
- ☑ Billions of parameters
- ☑ Based on Transformer architecture
- ☑ Understand context and semantics
- ☑ Can generate human-like text
- ☑ Can perform multiple tasks (Q&A, summarization, translation, code generation, etc.)
- ☑ Show emergent abilities with scale

3. How LLMs are Trained ?

Large Text Corpus
(Books, Websites, Articles, Code, etc.)
↓
Preprocessing
(Cleaning, Tokenization, Formatting)
↓
Self-Supervised Learning
(Next Token Prediction)
↓
Trained LLM
(Billions of Parameters)

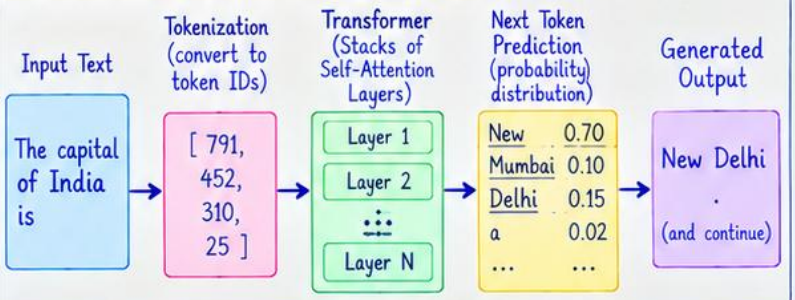
4. Training Objective (Next Token Prediction)

LLMs are trained to predict the next most likely token in a sequence.

Example:

Input: "The capital of India is"
Model predicts next token ⟶ "New"
Then ⟶ "Delhi"
Then ⟶ "." (and so on)

5. LLM Architecture (High Level)



6. Popular LLM Models

Model	Organization	Parameters (Approx)	Key Features
GPT-3.5 / GPT-4	OpenAI	175B+	General purpose, strong chat & reasoning
LLaMA 2 / 3	Meta	7B, 13B, 70B	Open source, efficient
Claude	Anthropic	Varies	Safe & helpful AI
Gemini	Google	Varies	Multimodal (text, image, video), strong reasoning
Mistral	Mistral AI	7B, 8x7B	Open source, high performance

@codedsoul_05

7. Types of LLMs

- **Base Model** → Trained only on general data (next token prediction).
- **Instruction-Tuned** → Fine-tuned to follow human instructions (e.g., ChatGPT).
- **Domain-Specific** → Trained/fine-tuned on specific domain (medical, legal, finance, etc.).
- **Multimodal** → Can understand multiple inputs (text, images, audio, video). (e.g., GPT-4V, Gemini)

8. Capabilities of LLMs

Text Generation	Summarization	Question Answering	Translation
Code Generation	Reasoning	Information Extraction	Image Understanding (multimodal)

9. Limitations of LLMs

- ☒ Can generate incorrect information (hallucinations)
- ☒ Knowledge is limited to training data (cutoff date)
- ☒ Not real-time (unless connected to tools)
- ☒ Can be biased (from training data)
- ☒ Struggles with complex calculations
- ☒ Lacks true understanding or common sense
- ☒ High computational and cost requirements

10. Key Takeaways

- ☑ LLMs predict the next token using Transformer architecture.
- ☑ Trained on massive text data.
- ☑ Can perform many language tasks.
- ☑ Different types for different use cases.
- ☑ Powerful but not perfect (know the limitations).
- ☑ Can be enhanced with tools like RAG, fine-tuning, and agents.

Learn ⟶ Understand ⟶ Build ⟶ Apply ⟶ Grow

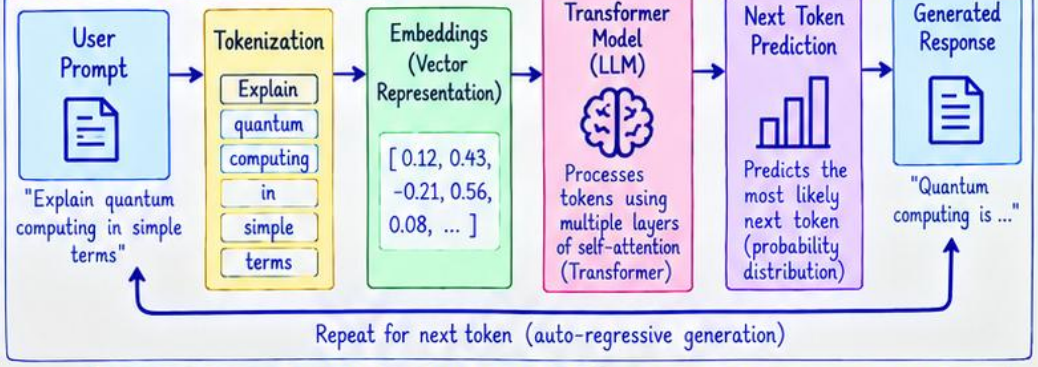
@codedsoul_05

7. How ChatGPT & LLMs Work

1. High Level Overview

- ChatGPT is a Large Language Model (LLM) based on the Transformer architecture.
- It is trained on massive text data to learn language patterns, context, and world knowledge.
- When you give a prompt, it processes the text, predicts the next token step by step, and generates a human-like response.
- It can also use tools (browsing, code interpreter, plugins, etc.) to enhance its responses.

2. End-to-End Workflow



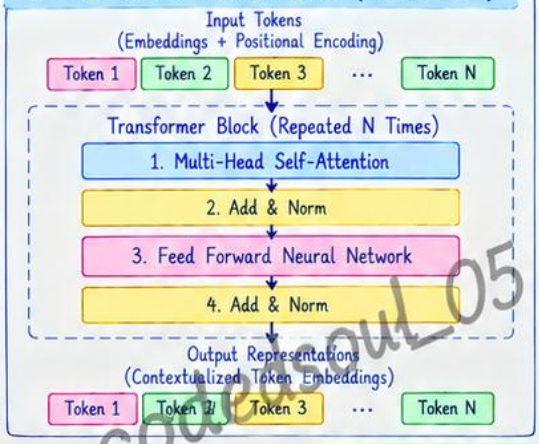
3. Tokenization

- Input text is split into small pieces called tokens.
- A token can be a word, part of a word, or even a single character.
- Each token is converted into a unique token ID and then into a vector (embedding).

Example: "ChatGPT is amazing!"



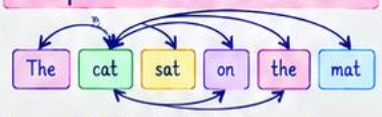
4. Transformer Architecture (Inside LLM)



5. Self-Attention (Key Idea)

- Each token looks at all other tokens in the input to understand context.
- It calculates Attention Scores to find which tokens are more relevant.
- This helps the model understand the meaning of a word based on the whole sentence.

Example: "The cat sat on the mat"



The word "cat" attends to other words (e.g., "sat", "mat") to understand meaning.

6. Next Token Prediction

- The model outputs a probability distribution over the next possible token.
- The most likely token is selected using techniques like:
  - ☑ Greedy Search (pick highest probability)
  - ☑ Top-k Sampling (choose from top k tokens)
  - ☑ Top-p (Nucleus Sampling)
  - ☑ Temperature (controls randomness)

Example (Simplified)

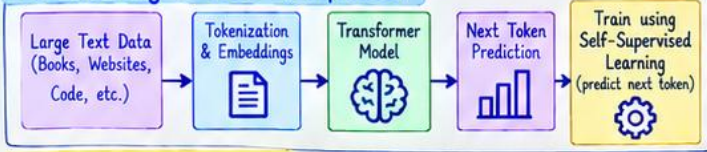
Input: "The sky is"

Next Token	Probability
blue	0.65
clear	0.20
bright	0.10
...	...

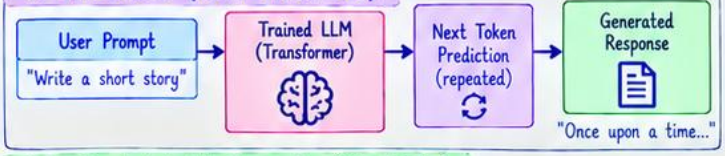
7. Decoding Strategies

Method	How it Works	Use Case
Greedy Search	Pick highest probability token	Deterministic, less creative
Top-k Sampling	Choose from top k tokens (e.g., k=50)	Better variety
Top-p (Nucleus)	Choose from top tokens with cumulative prob p (e.g., p=0.9)	More natural text
Temperature	Controls randomness (higher = more creative)	Creative vs focused responses

8. Training Process (Simplified)



9. Inference (Real-Time Use)



10. Key Takeaways

- ☑ ChatGPT uses a Transformer-based LLM.
- ☑ Input is tokenized and converted to vectors.
- ☑ Self-attention helps understand context.
- ☑ Model predicts the next token step by step.
- ☑ Decoding strategies control the output style.
- ☑ It can also use tools (RAG, browsing, code, etc.) for better responses.

11. Real World Example (ChatGPT)

User: "Explain quantum computing in simple terms."

ChatGPT: "Quantum computing uses the principles of quantum mechanics to process information in a fundamentally different way than classical computers. Instead of bits (0 or 1), it uses qubits which can be both 0 and 1 at the same time ..."

Prompt → Tokenization → Transformer (Self-Attention) → Next Token Prediction → Human-like Response

codedsoul_05

8. Prompt Engineering Basics

1. What is Prompt Engineering ?

- Prompt Engineering is the art of giving the right input (prompt) to an LLM to get the desired and high-quality output.
- It helps the model understand your intention, context and format clearly.
- A good prompt can significantly improve the accuracy, relevance and usefulness of the response.

2. Anatomy of a Prompt

Role	Defines who the model is (e.g., You are a teacher...)
Context	Provides background information
Task	Clearly states what to do
Format	Specifies output structure (e.g., bullet points, table, code)
Constraints	Adds rules or limitations (e.g., length, tone, style)

3. Basic Prompt Template

You are a <role>.

Context:
<provide background information>

Task:
<what you want the model to do>

Format:
<how you want the output>

Constraints:
<any specific rules>

4. Simple Example

Basic Prompt	Better Prompt (using template)
Tell me about climate change. →	You are a science teacher. Explain climate change in simple terms for a 10-year-old. Use bullet points and real-world examples.

5. Good vs Bad Prompts

✗ Bad Prompt	☑ Good Prompt
Write something about AI.	Explain the key applications of Generative AI with examples.
Explain this.	Explain this concept in simple terms with a real-world example.
Give me the code.	Write a Python function to find the factorial of a number with comments.
Summarize the article.	Summarize the article in 5 key points for a beginner.

6. Prompting Techniques (Beginner Level)

1. Zero-shot Prompting (directly ask)
2. Few-shot Prompting (with examples)
3. Role-based Prompting (assign a role)
4. Step-by-step Prompting (chain of thought)
5. Format-based Prompting (table, json, bullets, etc.)

7. Examples of Prompting Techniques

Technique	Prompt Example	Use Case
Zero-shot	Explain photosynthesis in simple terms.	Simple questions
Few-shot	Q: 2+2=? A: 4 Q: 5+3=? A:	Provide examples for better results
Role-based	You are a career advisor. Suggest career options for a science student.	Domain-specific expertise
Chain of Thought	Solve this step by step: If a train travels 120 km in 2 hours, what is its speed?	Complex reasoning problems
Format-based	List 5 benefits of exercise in a table with columns: Benefit | Description	Structured output

8. Role-based Prompting Examples

Teacher	You are a teacher. Explain machine learning in simple terms with examples.
Career Advisor	You are a career advisor. Suggest a learning path to become an AI engineer.
Content Writer	You are a content writer. Write a LinkedIn post about the importance of Generative AI.
Developer	You are a senior developer. Write a Python code to connect to OpenAI API.
Analyst	You are a data analyst. Analyze this data and provide key insights in a table.

9. Important Tips

- ☑ Be clear and specific.
- ☑ Provide enough context.
- ☑ Use a role for better results.
- ☑ Specify the output format.
- ☑ Add constraints (length, tone, style).
- ☑ Use examples when needed.
- ☑ Break complex tasks into steps.
- ☑ Iterate and refine the prompt.
- ☑ Avoid vague or ambiguous prompts.
- ☑ Test different phrasings to get the best result.

10. Real World Examples

Summarization
You are a research assistant. Summarize the following article in 5 key points: <article text>

Idea Generation
You are a business consultant. Suggest 5 business ideas in the health and wellness domain.

Code Generation
You are a Python expert. Write a function to check if a number is prime with comments.

Data Analysis
You are a data analyst. Analyze the following sales data and provide key insights in a table.

Good Prompt → Better Context → Clear Instructions → Useful Output

codedsoul_05

@codedsoul_05

9. Advanced Prompting — Zero/Few-shot, CoT, Roles

1. Why Advanced Prompting ?

- Helps get more accurate, relevant and high-quality responses.
- Guides the model to think step by step.
- Useful for complex tasks like reasoning, analysis, coding, planning, etc.
- Makes the model behave like an expert in a specific role or domain.
- Reduces ambiguity and unwanted outputs.

2. Types of Prompting Techniques

1. Zero-shot Prompting (ask directly)
2. Few-shot Prompting (with examples)
3. Chain of Thought (CoT) (step-by-step reasoning)
4. Role-based Prompting (assign a role)
5. Instruction-based Prompting (clear instructions)
6. Advanced Techniques (Refinement, Self-Consistency, etc.)

3. Zero-shot Prompting

- Give a direct instruction without any examples.
- Works well for simple and common tasks.

Example :

Prompt: Summarize the importance of clean drinking water.

Output (LLM):

Clean drinking water is essential for good health, prevents diseases, supports growth and development, and improves quality of life.

4. Few-shot Prompting

- Provide a few examples in the prompt.
- Helps the model understand the pattern and generate better results.

Example :

Prompt:
Classify the sentiment of the following reviews:

Review: "The product is excellent!" → Positive
Review: "Not good, very poor quality." → Negative
Review: "It is okay, could be better." → Neutral
Review: "I really loved it!" → ?

Output: Positive

5. Chain of Thought (CoT) Prompting

- Ask the model to think step by step before giving the final answer.
- Improves reasoning for complex problems (math, logic, planning, etc.).

Example :

Prompt: Solve this step by step:
If a train travels 120 km in 2 hours, what is its speed?

Output:

Step 1: Speed = Distance / Time
Step 2: Distance = 120 km
Step 3: Time = 2 hours
Step 4: Speed = 120 / 2 = 60 km/h
Final Answer: 60 km/h

6. Role-based Prompting

- Assign a specific role to the model (e.g., teacher, doctor, expert, etc.).
- Helps get domain-specific and structured responses.

Example :

Prompt: You are a financial advisor. Explain the benefits of mutual funds for beginners in simple terms.

Output:

Mutual funds allow you to invest in a diversified portfolio, are managed by professionals, help reduce risk, require low investment, and are suitable for long-term wealth creation.

7. Instruction-based Prompting

- Give clear and detailed instructions about the format, style, length, etc.
- Useful when you need structured output.

Example :

Prompt:
Write a 5-point list on how to stay healthy.
Format: use bullet points. Keep it under 50 words.

Output:

- Eat a balanced diet
- Exercise regularly
- Get enough sleep
- Stay hydrated
- Manage stress

8. Advanced Techniques

Technique	Description	Use Case
Self-Consistency	Generate multiple responses and choose the best.	Better reasoning and less error.
ReAct Prompting	Combine reasoning with action (use tools like search, code, etc.).	Real-world tasks.
Tree of Thoughts (ToT)	Explore multiple reasoning paths.	Complex problem solving.
Iterative Refinement	Ask the model to improve its own output.	Better and polished results.
Prompt Chaining	Break a big task into smaller steps.	Complex workflows.
Constrained Decoding	Add rules for output (e.g., JSON, length).	Structured output.

9. Comparison Example

Technique	Prompt	Output (Short)
Zero-shot	Explain AI.	General explanation.
Few-shot	Give 2 examples + classify.	Follows pattern correctly.
CoT	Solve step by step.	Detailed reasoning and correct answer.
Role-based	You are a teacher. Explain AI.	Simple and student-friendly explanation.
Instruction-based	Write in table format.	Structured table output.

10. Best Practices

- ☑ Be clear and specific.
- ☑ Provide enough context.
- ☑ Use examples when needed.
- ☑ Assign a role for better results.
- ☑ Break complex tasks into smaller steps.
- ☑ Specify the output format (table, JSON, bullets).
- ☑ Use CoT for reasoning tasks.
- ☑ Iterate and refine the prompt.
- ☑ Avoid ambiguous or vague instructions.
- ☑ Test different techniques and combine them.

11. Key Takeaways

- ☑ Advanced prompting helps unlock the full power of LLMs.
- ☑ Zero-shot: direct instruction.
- ☑ Few-shot: provide examples.
- ☑ CoT: step-by-step reasoning.
- ☑ Role-based: act like an expert.
- ☑ Use the right technique based on your task.

Clear Role → Good Context → Right Technique → Better Output

codedsoul_05

@codedsoul_05

10. LLM Parameters — Temperature, Tokens, Context

1. What are LLM Parameters ?

- Parameters (or settings) control how the model generates responses.
- They do not change the model's knowledge, but they influence the style, length, creativity, and quality of the output.
- Common parameters include temperature, max tokens, top-p, frequency penalty, etc.

2. Key Parameters Overview

Parameter	Purpose	Common Values
Temperature	Controls randomness (creativity vs. accuracy)	0.0 – 2.0 (default 0.7)
Max Tokens	Limits the length of the response	50 – 4096+ (model dependent)
Context Window	Maximum input + output tokens the model can consider	4K, 8K, 16K, 32K, 128K (model dependent)
Top-p (Nucleus)	Controls diversity by choosing from top probable tokens	0.0 – 1.0 (default 1.0)
Other Parameters	Frequency penalty, Presence penalty, Stop sequences, etc.	Varies

3. Temperature

- Controls the randomness of the output.
- Low value → more focused, accurate, and deterministic.
- High value → more creative, diverse, and unexpected.

Low Temperature (0.2)	High Temperature (1.0)
The capital of India is New Delhi.	Ah, the heart of India beats in New Delhi — a vibrant city full of history, culture and opportunities!

4. Temperature Examples

0.0	Deterministic, same output every time (less creative)
0.2	More focused and factual (best for Q&A, coding)
0.5	Balanced (default, good for general use)
0.8	More creative and diverse
1.0	Very creative, may generate unexpected or unusual text
1.5+	Highly random, can be off-topic or inconsistent

5. Max Tokens (Output Length)

- Specifies the maximum number of tokens the model can generate in a response.
- Helps control the length and cost.
- If the response reaches the max token limit, it will stop.

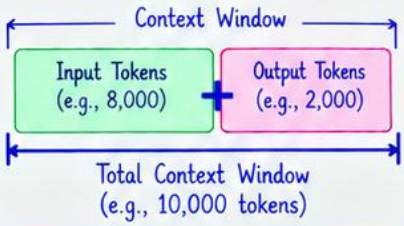
Example :

Prompt: Explain cloud computing.
Max tokens = 50 → short answer
Max tokens = 300 → detailed answer

Max Tokens = 50	Max Tokens = 300
Cloud computing provides computing resources over the internet ...	Cloud computing provides on-demand access to servers, storage, databases, networking ... [detailed explanation]

6. Context Window

- The total number of tokens (input + output) the model can consider at once.
- Larger context windows allow longer documents, conversations, and code.
- If the input exceeds the limit, older information may be truncated.



7. Top-p (Nucleus Sampling)

- Instead of choosing from all possible tokens, it selects from the top P cumulative probability.
- Helps control diversity similar to temperature.
- Commonly used with temperature for better results.

All Tokens (sorted by probability)	
Token A	0.40
Token B	0.25
Token C	0.15
Token D	0.10
Token E	0.05
Token F	0.05

→

Top-p = 0.9 (selects top tokens until cumulative = 0.9)

Token A	0.40
Token B	0.25
Token C	0.15
Token D	0.10

} Cumulative = 0.90

8. Other Useful Parameters

Parameter	Purpose	Example Values
Frequency Penalty	Reduces repetition of the same phrases	-2.0 to 2.0 (default 0.0)
Presence Penalty	Encourages new topics or ideas	-2.0 to 2.0 (default 0.0)
Stop Sequences	Specifies where the model should stop	e.g., ["\n", "END"]
Best of (n)	Generates multiple responses and picks the best	1 – 20
Logprobs	Returns probability of generated tokens	True / False

codedsoul_05

9. Parameter Tuning Tips

- ☑ Use low temperature (0.0 – 0.3) for factual answers, coding, and analysis.
- ☑ Use medium temperature (0.5 – 0.7) for balanced responses.
- ☑ Use high temperature (0.8 – 1.0) for creative writing, brainstorming.
- ☑ Adjust max tokens based on the required output length.
- ☑ Use larger context models (e.g., 32K, 128K) for long documents.
- ☑ Experiment with top-p and penalties for better control.
- ☑ Test different combinations to find the best results for your use case.

10. Real World Examples

Use Case	Recommended Settings	Why ?
Q&A (Factual)	Temperature: 0.0 – 0.3 Max tokens: 100 – 300	More accurate and reliable answers
Creative Writing	Temperature: 0.7 – 1.0 Max tokens: 300 – 800	More diverse and creative text
Code Generation	Temperature: 0.0 – 0.3 Max tokens: 200 – 500	Precise and correct code
Long Document Analysis	Large context (32K/128K) Max tokens: 500 – 2000	Handle long input and detailed output

11. Key Takeaways

- ☑ Temperature controls creativity vs. accuracy.
- ☑ Max tokens limits the response length.
- ☑ Context window defines how much input the model can handle.
- ☑ Top-p provides alternative way to control randomness.
- ☑ Other parameters (penalties, stop sequences) help refine output.
- ☑ Use the right combination based on your task.
- ☑ Experiment and test to get the best results.

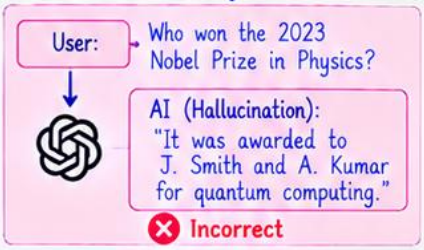
Right Parameters → Better Responses → Higher Productivity

@codedsoul_05

11. Hallucinations, Limitations & AI Safety

1. What are Hallucinations ?

- Hallucinations happen when an AI model generates information that is incorrect, misleading, or completely made up, but presented confidently as fact.
- The model does not intentionally lie, it predicts the most likely next words, which can sometimes result in false information.
- Hallucinations can look very real and convincing.



2. Why Do Hallucinations Happen ?

1. **Predicts next word, not facts**
   LLMs generate text based on patterns, not real-time truth checking.
2. **Training data limitations**
   If the training data is incomplete, outdated, or biased.
3. **Ambiguous prompts**
   Vague or unclear questions can lead to incorrect answers.
4. **Overconfidence**
   The model often gives confident answers even when uncertain.
5. **Complex reasoning**
   Difficult tasks (math, logic, multi-step reasoning) increase the chance of errors.

3. Examples of Hallucinations

Type	Example Output	Issue
Fake Facts	"The capital of Australia is Sydney."	Incorrect (factual error)
Fake Citations	"According to a 2022 study by Smith et al..."	Non-existent source
Wrong Numbers	"India's population is 2.5 billion."	Incorrect data
Invented People/Events	"Professor John K. Roberts discovered..."	Made-up information
Outdated Information	"The latest iPhone model is iPhone 14."	May be outdated
Misleading Explanation	"Quantum computing uses electrons instead of qubits."	Technically incorrect

4. Limitations of LLMs

- No real-time knowledge (cutoff date)
- Depends on training data (which may be biased or incomplete)
- May generate incorrect or made-up information
- Limited in complex math and precise calculations
- Can misunderstand context or nuanced questions
- Struggles with highly specialized domains without proper data
- May reflect biases present in training data
- Does not have consciousness, opinions, or real understanding

5. How to Identify Hallucinations ?

- ☑ Verify important facts from trusted sources.
- ☑ Look for citations and check if they exist.
- ☑ Be cautious with specific numbers, dates, and names.
- ☑ If an answer seems too confident or unusual, double-check.
- ☑ Ask follow-up questions or rephrase the prompt.
- ☑ Compare answers from multiple sources.
- ☑ Use AI as a starting point, not the final authority.

6. AI Safety — Key Areas

- **Reliability** — Reduce hallucinations and improve factual accuracy.
- **Security** — Prevent misuse, data leakage, and attacks (prompt injection).
- **Bias & Fairness** — Avoid harmful, biased, or discriminatory outputs.
- **Privacy** — Protect user data and de-identify sensitive information.
- **Transparency** — Clearly communicate AI's capabilities and limitations.
- **Responsible Use** — Use AI for good and avoid harmful or illegal activities.

7. Strategies to Reduce Hallucinations

- Give clear and specific prompts.
- Use trusted and up-to-date sources (via tools).
- Ask the model to cite sources.
- Break complex questions into smaller steps.
- Use techniques like Retrieval Augmented Generation (RAG) to provide real documents.
- Validate outputs for critical use cases.

codedsoul_05

8. Real World Risks

Area	Risk Example	Impact
Healthcare	Incorrect medical advice	Can harm health
Finance	Wrong investment information	Financial loss
Education	False historical facts	Misleading learning
Legal	Incorrect legal interpretation	Serious consequences
Business	Wrong market data/analysis	Poor decisions
Personal Use	Made-up product information	Wasted time and money

9. Best Practices for Safe Use

- ☑ Use AI as an assistant, not a decision maker.
- ☑ Always verify critical information.
- ☑ Be aware of its limitations.
- ☑ Avoid sharing sensitive or personal data.
- ☑ Use official and trusted tools.
- ☑ Follow company/organization guidelines.
- ☑ Educate yourself and others about responsible AI use.
- ☑ Combine AI with human expertise.

10. Key Takeaways

1. Hallucinations are a real limitation of current LLMs.
2. AI can be very helpful, but it can also be wrong.
3. Always verify important information.
4. Use AI responsibly and safely.
5. Understand the risks and follow best practices.
6. AI is a powerful tool when used with awareness and caution.

Use AI Responsibly → Verify → Be Aware → Stay Safe → Get Real Value

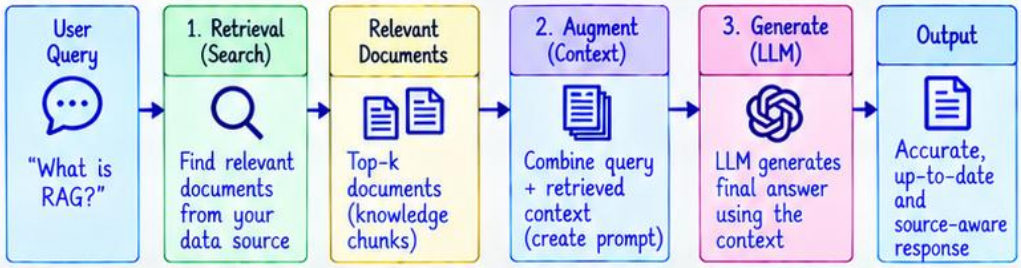
@codedsoul_05

12. RAG — Retrieval Augmented Generation

1. What is RAG ?

- RAG (Retrieval Augmented Generation) combines a retrieval system (search) with a generative LLM.
- Instead of relying only on the model's internal knowledge, it fetches relevant information from external sources (documents, databases, websites, etc.) and uses that context to generate a more accurate and up-to-date response.
- Useful for domain-specific, real-time, and private knowledge where the model alone may not have the information.

2. How RAG Works (Workflow)



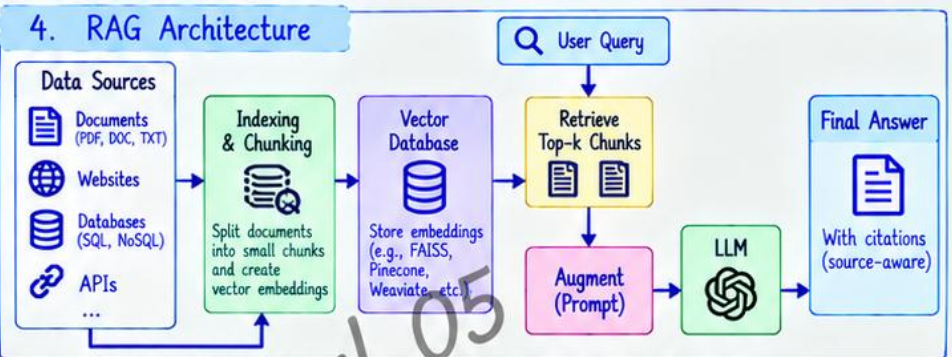
Flow

User Query → Retrieve → Top-k Documents → Augment (Prompt) → LLM → Final Answer

3. Why Use RAG ?

- ☑ Provides up-to-date information.
- ☑ Reduces hallucinations.
- ☑ Works with private or proprietary data (e.g., company documents).
- ☑ Cites sources for transparency.
- ☑ More accurate for domain-specific questions.
- ☑ Can handle large amounts of external knowledge without retraining the model.

4. RAG Architecture



5. Document Processing (Indexing)

1. **Load Documents** (PDF, DOC, HTML, etc.)
2. **Split into Chunks** (e.g., 500–1000 tokens with overlap)
3. **Create Embeddings** (using models like OpenAI, BGE, etc.)
4. **Store in Vector Database** (FAISS, Pinecone, Weaviate, etc.)

6. Retrieval Methods

Method	Description
Semantic Search	Finds meaning-similar content using vector embeddings.
Keyword Search (BM25)	Finds exact keyword matches.
Hybrid Search	Combines semantic + keyword search (better results).
Re-ranking	Re-ranks retrieved chunks using a smaller model for higher relevance.

7. Chunking Strategies

Strategy	When to Use
Fixed Size	Simple and fast (e.g., 500 tokens) for most use cases.
Sliding Window	Overlapping chunks for better context continuity.
Semantic Chunking	Split based on meaning (e.g., headings, paragraphs).
Structure-based	Use document structure (titles, sections, tables, etc.).

8. Example (Simplified)

User Query

"What are the benefits of Tata AIA Health Secure Pro?"

Retrieved Context (from documents)

1. Covers hospitalization
2. Pre & post hospitalization
3. Wellness benefits
4. Tax benefits
... ...

Final Answer (LLM)

Tata AIA Health Secure Pro offers hospitalization cover, pre & post hospitalization benefits, wellness benefits and tax benefits under Section 80D. [Source: Tata AIA Product Brochure]

9. Tools & Libraries

Purpose	Tools/Frameworks
Embeddings	OpenAI, HuggingFace (BGE, SBERT)
Vector Database	FAISS, Pinecone, Weaviate, Qdrant
Frameworks	LangChain, LlamaIndex (Haystack)
Re-ranking	Cohere Rerank, BGE Reranker
Document Loaders	LangChain Document Loaders
Others	OpenAI, Azure, AWS Bedrock

10. Use Cases

- ☑ Company document Q&A
- ☑ Customer support chatbot
- ☑ Research and summarization
- ☑ Legal/financial document analysis
- ☑ Healthcare/insurance knowledge base
- ☑ Internal knowledge assistants
- ☑ Real-time information (news, policies)
- ☑ Educational assistants

11. Challenges & Best Practices

Challenges	Best Practices
⚠ Poor retrieval quality (irrelevant results)	☑ Use good chunking and metadata.
⚠ Large document handling	☑ Try hybrid search + re-ranking.
⚠ Chunking strategy selection	☑ Keep documents up-to-date.
⚠ Keeping data updated	☑ Use citations and source links.
⚠ Latency and cost	☑ Evaluate and tune retrieval parameters (top-k, chunk size).

12. Key Takeaways

- ☑ RAG combines retrieval (search) + generation (LLM).
- ☑ Improves accuracy, reduces hallucinations.
- ☑ Works with external, up-to-date and private data.
- ☑ Key components: data → embeddings → vector DB → retrieval → augment → LLM.
- ☑ Useful for real-world applications across industries.

Better Retrieval → Better Context → More Accurate Answers → Real-World Value

codedsoul_05

@codedsoul_05

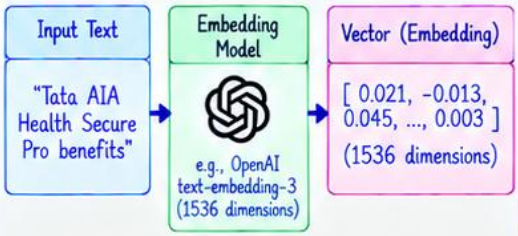
13. Embeddings & Vector Databases

1. What are Embeddings ?

- Embeddings are numerical representations (vectors) of text, images, or other data that capture their meaning.
- Similar items have similar vectors (are close in vector space).
- They allow machines to understand semantic meaning, not just exact keywords.
- Generated using models like OpenAI (text-embedding-3), Sentence-BERT, etc.

Example : "king", "queen" and "royal" have similar vectors, so they are close in the embedding space.

2. How Embeddings Work ?

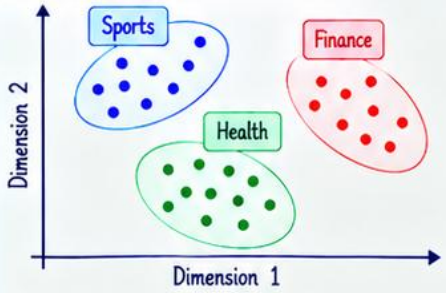


This vector captures the meaning of the text, not just the words.

3. Properties of Embeddings

Property	Description
Dense vectors	Fixed-length numerical arrays (e.g., 1536 dimensions).
Semantic meaning	Capture context and meaning of the data.
Similarity	Similar items have vectors close to each other.
Generalization	Can find related content even with different words.
Model-dependent	Quality depends on the embedding model used.

4. Visualizing Embeddings



Similar items are closer in the vector space, and different topics form different clusters.

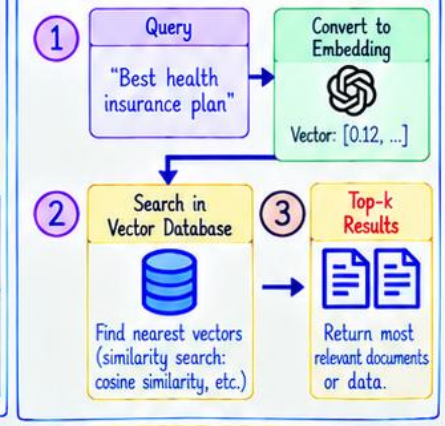
5. What are Vector Databases ?

- Vector databases store and search high-dimensional embeddings efficiently.
- They enable fast similarity search to find the most relevant data.
- Commonly used in RAG, recommendation systems, semantic search, and more.

Popular Vector Databases

Pinecone Fully managed, scalable, easy to use.	**FAISS** Open-source by Meta, very fast.	Open-source, rich features.
Qdrant Open-source, high performance.	**Milvus** Open-source, scalable.	Open-source, simple and lightweight.

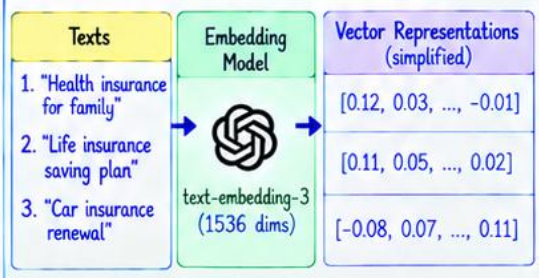
6. How Vector Search Works ?



7. Similarity Measures

Method	Formula / Idea	Use Case
Cosine Similarity	$\cos(\theta) = \frac{A \cdot B}{\|\|A\|\|\|\|B\|\|}$	Most common for text embeddings.
Euclidean Distance	$d = \sqrt{\sum(A - B)^2}$	Used in some vector databases.
Dot Product	$A \cdot B$	Fast and simple, often used in practice.
Manhattan Distance	$d = \sum\|A - B\|$	Less common, for specific use cases.

8. Example: Text to Embeddings



9. Use Cases

- ☑ RAG (Retrieval Augmented Generation)
- ☑ Semantic search (documents, FAQs)
- ☑ Chatbots with company knowledge
- ☑ Recommendation systems (products, videos, content)
- ☑ Image similarity search
- ☑ Personalized marketing
- ☑ Fraud detection (similar patterns)
- ☑ Legal and medical document search
- ☑ Enterprise knowledge management

10. Code Example (Python)

```python
from openai import OpenAI
from pinecone import Pinecone

# 1. Create embedding
client = OpenAI(api_key="YOUR_KEY")
text = "Tata AIA Health Secure Pro benefits"
response = client.embeddings.create(
    model="text-embedding-3-small",
    input=text
)
embedding = response.data[0].embedding

# 2. Store in Pinecone (example)
pc = Pinecone(api_key="YOUR_KEY")
index = pc.Index("my-index")
index.upsert([("doc1", embedding)])
```

11. Key Takeaways

- ☑ Embeddings convert data into numerical vectors that capture meaning.
- ☑ Vector databases store and search embeddings efficiently.
- ☑ Similarity search helps find relevant information, even with different words.
- ☑ Used widely in RAG, search, recommendations, and knowledge systems.
- ☑ Choose the right embedding model and vector database based on your use case.
- ☑ Understand similarity metrics and indexing for better performance.

12. Comparison of Vector Databases

Database	Type	Best For	Key Features
Pinecone	Managed	Production apps	Scalable, easy API, reliable
FAISS	Open-source	Research & large-scale	Very fast, no server
Weaviate	Open-source	Enterprise & rich features	Hybrid search, metadata, GraphQL
Qdrant	Open-source	High performance	Filtering, payloads, scalable
Milvus	Open-source	Big data & AI	Scalable, high performance
Chroma	Open-source	Beginner friendly	Simple, lightweight

Embeddings → Store in Vector DB → Similarity Search → Relevant Results

@codedsoul_05

14. LangChain — Components & Architecture

1. What is LangChain ?

- LangChain is an open-source framework to build applications with LLMs.
- It helps connect LLMs with external tools, data sources, and memory to create powerful real-world AI applications.
- It provides ready-to-use components, integrations, and chains to simplify development.

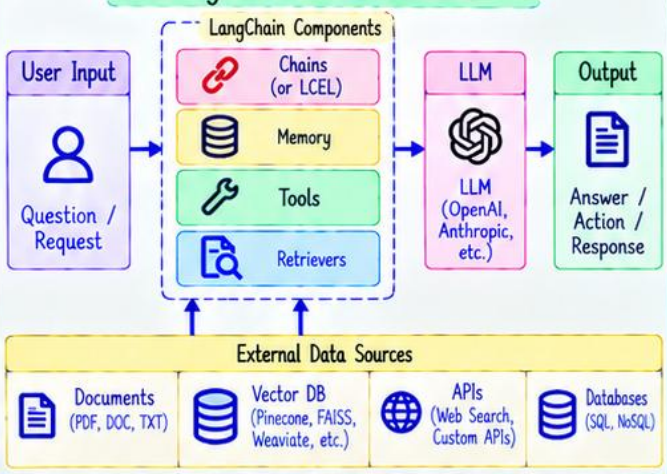
Example :

A chatbot that can answer questions from your company documents using LangChain.

2. Key Features

- ☑ Easy integration with LLMs (OpenAI, Anthropic, etc.).
- ☑ Connects to external data sources (documents, databases, APIs).
- ☑ Supports memory, tools, and agents.
- ☑ Modular and extensible.
- ☑ Large ecosystem and community.
- ☑ Works with different frameworks (e.g., RAG, Agents, Chains).
- ☑ Useful for both simple and complex AI applications.

3. High-Level Architecture



4. Core Components

Component	Purpose
LLMs	Generate text, reasoning and responses.
Prompts	Template and manage prompts.
Chains (LCEL)	Combine multiple steps into a workflow.
Memory	Store and recall past conversations.
Retrievers	Fetch relevant data from external sources.
Tools	Allow LLMs to interact with external systems (APIs, DBs).
Agents	Decide which tools to use and take actions.
Callbacks	Monitor, log and handle events during execution.

5. LangChain Workflow (RAG Example)

1. Load Documents — PDF, Web pages, Database, etc.
2. 2. Split Text — Text splitter (split into chunks)
3. Create Embeddings — Using OpenAI, HuggingFace, etc.
4. Store in Vector DB — Pinecone, FAISS, Weaviate, Chroma, etc.
5. Retrieve Relevant Chunks — Similarity search to find relevant data.
6. Create Prompt — Combine user query + retrieved context (using prompt template).
7. LLM Generate Answer — LLM (e.g., GPT-4o) generates final response.
8. Output to User — Accurate, context-aware answer with sources (optional).

6. LCEL (LangChain Expression Language)

- LCEL is a modern, simple and flexible way to build chains in LangChain.
- It allows chaining components using the '|' operator.
- Easier to read, write, and maintain compared to older chain methods.

Example : Simple LCEL Chain

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(model="gpt-4o")
prompt = ChatPromptTemplate.from_template(
    """Answer the question: {question}"""
)

chain = prompt | llm
response = chain.invoke({"question": "What is AI?"})
print(response.content)
```

7. Tools in LangChain

- Tools allow LLMs to take actions outside the model (APIs, web search, calculations, databases, etc.).
- Useful for real-time information and task automation.

Example : Web Search Tool

```
from langchain_community.tools import DuckDuckGoSearchRun

search = DuckDuckGoSearchRun()
result = search.run("Latest AI news")
print(result)
```

8. Agents in LangChain

- Agents use LLMs to decide which tools to use and in what order.
- They can perform multi-step tasks (e.g., search, calculate, fetch data, etc.).

Example : Simple Agent

```
from langchain.agents import initialize_agent, Tool
from langchain_openai import ChatOpenAI
from langchain_community.tools import DuckDuckGoSearchRun

llm = ChatOpenAI(model="gpt-4o")
tools = [DuckDuckGoSearchRun()]

agent = initialize_agent(tools, llm, agent="zero-shot-react-description", verbose=True)
response = agent.run("Find latest AI news and summarize it.")
print(response)
```

9. Popular Integrations

Integration	Use Case
OpenAI	LLMs & embeddings
Anthropic	Claude models
HuggingFace	Open-source models & embeddings
Pinecone	Vector database
FAISS	Local vector search
Weaviate	Vector database
Chroma	Lightweight vector DB
SQL/NoSQL DBs	Structured data
APIs (SerpAPI, etc.)	Web search, real-time data

10. Use Cases

- ☑ Document Q&A (RAG)
- ☑ Chatbots with company data
- ☑ Research assistants
- ☑ Customer support automation
- ☑ Data analysis with databases
- ☑ Multi-tool AI agents
- ☑ Knowledge management systems
- ☑ Personal productivity assistants
- ☑ Enterprise search solutions

11. Key Takeaways

- ☑ LangChain connects LLMs with data, tools, and memory.
- ☑ It simplifies building real-world AI applications.
- ☑ Understand core components: Chains, Memory, Tools, Retrievers, Agents.
- ☑ Use LCEL for simple and powerful chains.
- ☑ Ideal for RAG, agents, and enterprise use cases.

LLM + External Data + LangChain Components = Powerful AI Applications

@codedsoul_05

15. AI Agents & Tool Calling

1. What are AI Agents ?

- AI Agents are LLM-powered systems that can perceive, plan, take actions (using tools), and achieve goals autonomously.
- They can interact with external systems (APIs, databases, websites, files, etc.) through tool calling.
- Unlike a standard chatbot, an agent use decide what to do, call tools, use the results, and continue the task until the goal is completed.

Example :

Plan a trip → search flights → check weather → create itinerary → send email.

2. Key Components of an Agent

Component	Purpose
LLM (Brain)	Understands the goal, reasons, and decides next action.
Tools	Functions/APIs the agent can call (e.g., web search, database, code execution, email, etc.).
Planner	Breaks the goal into smaller steps and decides which tool to use.
Memory	Stores conversation history, past actions, and results.
Guardrails	Enforces safety, permissions, and limits.
Executor	Calls the tool, gets the result, and sends it back to the LLM.
Output	Final answer or completed task.

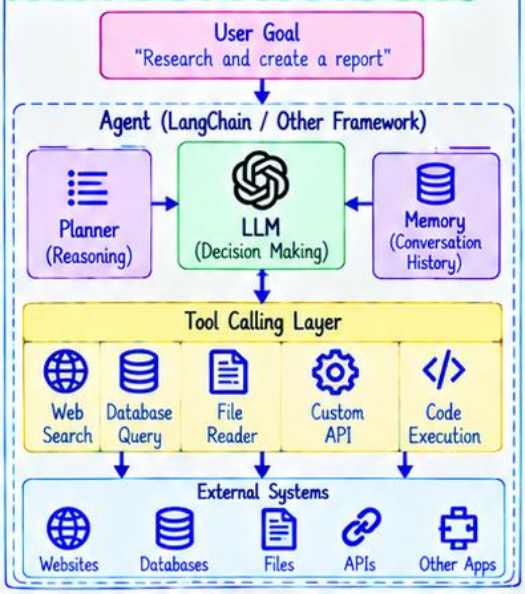
3. How Tool Calling Works ?



4. Types of Tools (Examples)

Tool Type	Examples
Web Search	Google Search, SerpAPI, Bing Search
APIs	Weather API, Flight API, Maps API, CRM API
Databases	PostgreSQL, MongoDB, Vector DB
File Tools	Read/Write files, PDFs, Excel, CSV
Code Execution	Python REPL, data analysis
Productivity	Email (Gmail), Calendar, Slack, Notion
Custom Tools	Your own business logic (e.g., insurance premium calculator)

5. Agent Architecture (High Level)



6. Example: Tool Calling with LangChain

```python
from langchain.agents import initialize_agent, Tool
from langchain.llms import ChatOpenAI
from langchain.tools import SerpAPIWrapper

# Define tools
tools = [
    Tool(
        name="Search",
        func=SerpAPIWrapper().run,
        description="Search the web for latest information"
    )
]

# Create agent
llm = ChatOpenAI(model="gpt-4o")
agent = initialize_agent(
    tools, llm,
    agent="zero-shot-react-description",
    verbose=True
)

# Run
response = agent.run("Find top 3 hotels in Delhi")
print(response)
```

7. Agent Types

Type	Description
ReAct Agent	Uses reasoning + actions (iterative tool calling).
Plan-and-Execute	Creates a plan first, then executes step by step.
OpenAI Functions	Uses function calling (built-in tool calling).
Self-Refine Agent	Iteratively improves results using feedback.
Multi-Agent	Multiple agents working together (e.g., researcher, writer, reviewer).

8. Real World Examples

- **Travel Assistant** — Search flights, hotels, create itinerary.
- **Shopping Assistant** — Find products, compare prices.
- **Data Analyst Agent** — Query databases, analyze data.
- **Research Agent** — Search, summarize papers, create report.
- **Email Assistant** — Read emails, draft and send replies.
- **Business Agent** — CRM updates, client follow-ups, lead generation.

9. Benefits

- ☑ Can perform real tasks, not just answer questions.
- ☑ Connects with real-world data and systems.
- ☑ Saves time and automates workflows.
- ☑ Improves accuracy with up-to-date information.
- ☑ Useful for business, research, productivity, and customer support.

10. Challenges

- ⚠ Tool reliability (API failures).
- ⚠ Security risks (malicious tools).
- ⚠ Potential for incorrect actions.
- ⚠ Needs good error handling.
- ⚠ Can be slower than simple Q&A.
- ⚠ Requires proper permissions and guardrails.

11. Key Takeaways

- ☑ AI Agents use LLMs + tools to complete real-world tasks.
- ☑ Tool calling allows interaction with external systems.
- ☑ LangChain provides easy framework to build agents.
- ☑ Useful for automation, research, customer support, and more.
- ☑ Always use safety measures, permissions, and monitoring.

Plan → Use Tools → Take Actions → Get Real Results

@codedsoul_05

16. LangGraph & Agentic AI

1. What is LangGraph ?

- LangGraph is a framework (by LangChain) for building stateful, multi-step, agent workflows using graphs.
- It lets you define nodes (steps) and edges (flow) to create reliable, controllable, and long-running agents.
- Useful for complex agents that need planning, tool calling, memory, human in the loop, and conditional logic.

Key Idea:
Represent an agent as a graph of nodes instead of a linear chain.

2. Why Use LangGraph ?

- ☑ Graph-based control flow (multi-step).
- ☑ State management (memory across steps).
- ☑ Conditional branching and loops.
- ☑ Tool calling and multi-agent workflows.
- ☑ Human-in-the-loop (approval checkpoints).
- ☑ Better observability and debugging.
- ☑ Suitable for production-ready agentic AI applications.

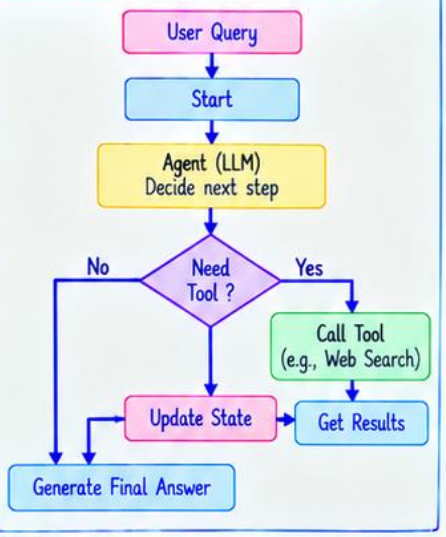
3. LangChain vs LangGraph

Feature	LangChain (Chains)	LangGraph
Flow Type	Linear (sequence)	Graph (nodes & edges)
State Management	Limited	Built-in, persistent state
Control Flow	Simple	Complex (branching, loops, conditional)
Best For	Simple workflows	Advanced agent workflows
Human in the Loop	Limited	Supported (checkpoints)
Use Case	Chatbots, simple RAG	Multi-step agents, tool use, long-running tasks

4. Key Concepts in LangGraph

Concept	Description
Node	A step in the workflow (e.g., LLM call, tool call, logic).
Edge	Defines the flow between nodes (can be conditional).
State	Shared data that flows across nodes (dict/TypedDict).
Graph	Collection of nodes and edges defining the workflow.
Conditional Edge	Routes to different nodes based on a condition (e.g., if tool needed).
Checkpoint	Pause for human approval (human-in-the-loop).
Tool Node	Node that calls external tools (search, DB, API, etc.).

5. Simple LangGraph Workflow



6. Example: LangGraph Agent (Python)

```python
from langgraph.graph import StateGraph, END
from typing import TypedDict
from langchain_openai import ChatOpenAI
from langchain.tools import Tool

# Define state
class AgentState(TypedDict):
    messages: list
    next_step: str

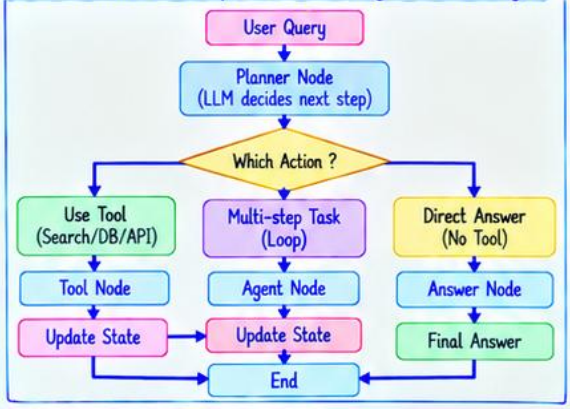
# Initialize LLM and tool
llm = ChatOpenAI(model="gpt-4o-mini")
search_tool = Tool(
    name="search",
    func=lambda q: "Web search result for: " + q,
    description="Search the web"
)

# Define nodes
def agent_node(state: AgentState):
    response = llm.invoke(state["messages"])
    return {"messages": state["messages"] + [response]}

# Build graph
workflow = StateGraph(AgentState)
workflow.add_node("agent", agent_node)
workflow.set_entry_point("agent")
workflow.add_edge("agent", END)
graph = workflow.compile()

# Run
result = graph.invoke({"messages": ["What is LangGraph?"]})
print(result["messages"][-1].content)
```

7. Advanced Example (Tool Calling + Branching)



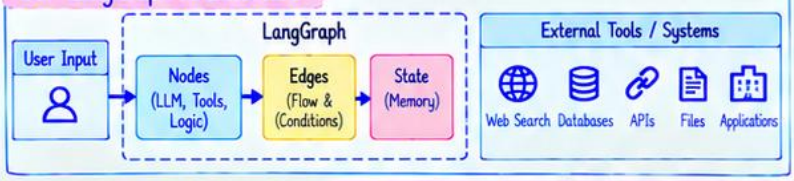
8. Agentic AI Patterns with LangGraph

Pattern	Use Case
ReAct Agent (tool use)	Search, code execution, data retrieval.
Plan-and-Execute	Break complex tasks into steps and execute.
Self-Refine	Iteratively improve answers.
Multi-Agent	Multiple specialized agents (e.g., researcher, writer).
Human-in-the-Loop	Add approval checkpoints.
RAG Agent	Combine retrieval with agentic reasoning.
Long-Running Agent	Automate tasks over time (e.g., monitoring, research).

9. Real-World Use Cases

- ☑ AI research assistant (search, summarize, report).
- ☑ Customer support agent (use company data + tools).
- ☑ Travel planner (flights, hotels, itinerary).
- ☑ Data analysis agent (query DB, generate charts).
- ☑ Enterprise knowledge assistant (RAG + tools).
- ☑ Multi-agent systems (different agents for different tasks).
- ☑ Automated workflow agent (email, calendar, CRM).

10. LangGraph Architecture



11. Best Practices

- ☑ Define clear state schema.
- ☑ Use modular nodes (LLM, tools, logic).
- ☑ Add error handling and fallback paths.
- ☑ Use checkpoints for human approval.
- ☑ Keep prompts and tools well-defined.
- ☑ Monitor and log agent execution.
- ☑ Test with real-world data and edge cases.

12. Key Takeaways

1. LangGraph enables graph-based agent workflows.
2. Supports state, branching, tool calling, and human in the loop.
3. More powerful than linear chains.
4. Ideal for complex, real-world agentic AI.
5. Works well with LangChain ecosystem.
6. Enables reliable, scalable, and production-ready agents.

Graph + State + Tools + LLM = Powerful Agentic AI

@codedsoul_05

17. Fine-Tuning, LoRA & PEFT

1. What is Fine-Tuning ?

- Fine-tuning means further training a pre-trained LLM on a smaller, domain-specific dataset to make it better at a particular task.
- It adapts the model's knowledge and style to your use case (e.g., customer support, medical, legal).
- Compared to training from scratch, fine-tuning is faster, cheaper and requires less data.

Example :

Fine-tune a general LLM (like Llama 3) on your company's FAQs to get accurate, domain-specific answers.

2. Types of Model Adaptation

Method	What is it ?
Full Fine-Tuning	Update all model parameters on new data.
Feature-based (Prompting)	No training, just better prompts (zero/few-shot).
Parameter-Efficient Fine-Tuning (PEFT)	Update only a small subset of parameters (e.g., LoRA, adapters, prefix-tuning).
LoRA (Low-Rank Adaptation)	A popular PEFT method that adds small trainable matrices to existing weights.
QLoRA	Memory-efficient LoRA with 4-bit quantization.
Adapter Tuning	Small trainable adapter layers added to each transformer block.
Prefix / Prompt Tuning	Train only continuous prompt embeddings (prefix tokens).

3. Full Fine-Tuning vs PEFT

Aspect	Full Fine-Tuning	PEFT (LoRA/Adapters)
Parameters Updated	All parameters (billions)	Only a small subset (millions)
Compute Requirement	High (GPUs/TPUs)	Low to medium
Memory Usage	Very high	Much lower
Training Time	Long (hours/days)	Short (minutes/hours)
Storage Size	Large (full model)	Small (adapter/LoRA weights only)
Performance	Best for large domain shift	Close to full fine-tuning for many use cases
Use Case	When you need maximum accuracy	Most real-word applications (cost-effective)

4. What is LoRA ?

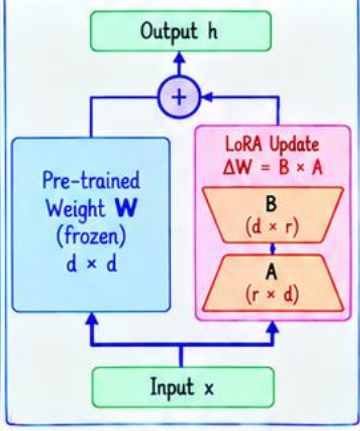
- LoRA (Low-Rank Adaptation) is a PEFT method that adds small, trainable low-rank matrices to the existing model weights.
- The original model weights remain frozen.
- It significantly reduces the number of trainable parameters, memory usage, and training cost.

Key Idea :

Instead of updating the full weight matrix W, we learn a small low-rank update $\Delta W = B \times A$

- $A \in \mathbb{R}^{r \times d}$ (down projection)
- $B \in \mathbb{R}^{d \times r}$ (up projection)
- r = low rank (e.g., 8, 16, 32)

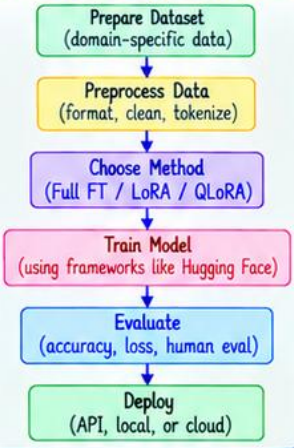
5. LoRA Diagram (Simplified)



6. PEFT Methods Overview

Method	Description
LoRA	Low-rank matrices added to attention weights.
QLoRA	LoRA + 4-bit quantization (less memory).
Adapters	Small trainable layers in each transformer block.
Prefix Tuning	Train continuous prefix embeddings.
Prompt Tuning	Train soft prompt tokens (input embeddings).
IA^3	Learn sparse scaling vectors for attention and FFN.
BitFit	Train only bias parameters (very few).

7. Fine-Tuning Workflow



8. Example: LoRA Fine-Tuning with Hugging Face

```python
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model, TaskType

model_name = "meta-llama/Meta-Llama-3-8B"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name, load_in_4bit=True, device_map="auto"
)

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type=TaskType.CAUSAL_LM
)
model = get_peft_model(model, lora_config)
# Train using your dataset (e.g., SFT with Trainer)
```

9. Dataset Preparation Tips

- ☑ Use high-quality, relevant, and clean data.
- ☑ Follow instruction format (instruction → input → output).
- ☑ Remove irrelevant or sensitive data.
- ☑ Use diverse examples and edge cases.
- ☑ Split into train/validation/test sets.
- ☑ For chat models, use conversation format (e.g., Alpaca, ShareGPT).

Example Format (Instruction Tuning)

```
{
  "instruction": "What is term insurance?",
  "input": "",
  "ootput": "Term insurance is a life insurance
   plan that provides financial protection..."
}
```

10. Evaluation Metrics

Metric	What it Measures
Perplexity (PPL)	How well the model predicts text (lower is better).
Accuracy	Correctness on task-specific data.
BLEU / ROUGE	Text generation quality (summarization).
Human Evaluation	Real-world usefulness and safety.
Domain-specific Metrics	Task-specific (e.g., exact match, F1 score for QA).

11. Best Practices

- ☑ Start with a good pre-trained model.
- ☑ Use PEFT (LoRA/QLoRA) for cost-effective training.
- ☑ Keep dataset high-quality and well-formatted.
- ☑ Tune hyperparameters (learning rate, batch size, rank r).
- ☑ Monitor for overfitting (use validation set).
- ☑ Use safety filters and red-teaming.
- ☑ Evaluate with real user scenarios.
- ☑ Deploy with monitoring and continuous improvement.

12. Common Use Cases

- ☑ Domain-specific chatbots (e.g., insurance, healthcare, legal).
- ☑ Customer support automation.
- ☑ Internal knowledge assistants (company documents).
- ☑ Summarization and report generation.
- ☑ Industry-specific Q&A (finance, real estate, travel).
- ☑ Multi-lingual and regional language support.
- ☑ Style and tone adaptation (brand voice).

13. Key Takeaways

- ☑ Fine-tuning adapts LLMs to your domain.
- ☑ LoRA and other PEFT methods make fine-tuning practical and affordable.
- ☑ PEFT gives good performance with fewer resources.
- ☑ Use high-quality data and proper evaluation.
- ☑ Choose the right method based on your use case, budget, and performance needs.
- ☑ Fine-tuning + RAG can give even better results for enterprise applications.

Adapt Pre-trained Models → Train Efficiently → Domain-specific Intelligence → Real-World Impact

@codedsoul_05

18. Multimodal AI + Open-Source LLMs

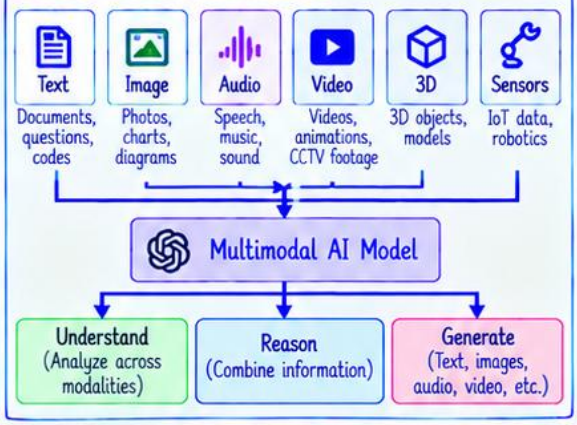
1. What is Multimodal AI ?

- Multimodal AI can understand and work with multiple types of data such as text, images, audio, video, and more.
- It can take inputs from different modalities, combine the information, and generate useful outputs.
- It is closer to human intelligence because humans also learn from many types of information (see, hear, read, etc.).

Example :

You can upload an image, ask a question in text, and get a detailed answer.

2. Modalities in AI



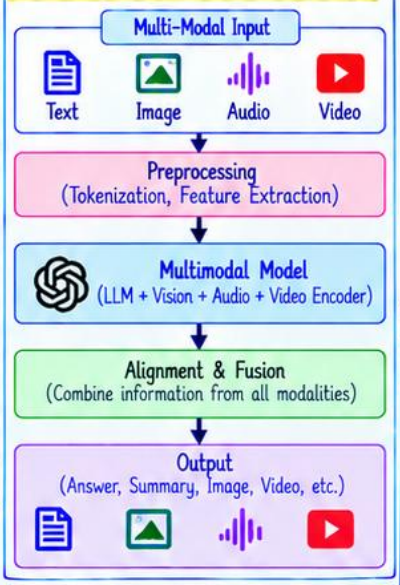
3. Real-World Use Cases

Healthcare	Analyze medical images + patient records
Education	Understand text, images, and videos for better learning
E-commerce	Product search using images (e.g., upload product photo)
Autonomous Vehicles	Process camera, sensor, and map data
Security & Surveillance	Analyze video + audio for real-time alerts
Robotics	Understand visual, audio, and sensor data
Business Intelligence	Analyze documents, charts, and meetings (audio)

4. Popular Multimodal AI Models

Model	Organization	Modalities	Key Features
GPT-4o	OpenAI	T + 🖼 ♫ ▶	Text, image, audio, video (real-time)
Gemini 1.5	Google	T 🖼 ♫ ▶	Large context, multimodal, video understanding
Claude 3	Anthropic	T 🖼 ♫ ▶	Vision + text, strong reasoning
Llama 3.2 (vision)	Meta	T 🖼 ♫ ▶	Open-source, text + image
Qwen-VL	Alibaba	T 🖼 ♫ ▶	Open-source, multimodal (text, image)
Phi-3 Vision	Microsoft	T 🖼 ♫ ▶	Lightweight, multimodal
BLIP-2	Salesforce	T 🖼 ♫ ▶	Image + text understanding
DeepSeek-VL	DeepSeek	T 🖼 ♫ ▶	Open-source, multimodal

5. How Multimodal AI Works ?



6. What are Open-Source LLMs ?

- Open-source LLMs are large language models whose model weights (and often training code) are publicly available.
- Anyone can download, use, modify, and build on them.
- They help researchers, developers, and businesses create custom AI solutions without high cost.

Example :

Llama (Meta), Mistral, Gemma (Google), Phi (Microsoft), Qwen (Alibaba), DeepSeek.

7. Popular Open-Source LLMs

Model	Organization	Size (Params)	Key Features
Llama 3	Meta	8B, 70B	High quality, widely used
Mistral	Mistral AI	7B, 8x7B	Efficient, high performance
Gemma	Google	2B, 7B	Lightweight, safe
Phi-3	Microsoft	3.8B, 7B	Small, efficient
Qwen	Alibaba	1.8B – 72B	Strong multilingual
DeepSeek	DeepSeek	7B, 67B	Competitive performance

8. Multimodal vs Text-Only LLMs

Feature	Multimodal AI	Text-Only LLMs
Input Types	Text, images, audio, video, etc.	Only text
Understanding	Cross-modal understanding	Text understanding only
Use Cases	More diverse (real-world tasks)	Limited to text tasks
Examples	GPT-4o, Gemini, Claude, Llama 3.2 (vision)	Llama 3, Mistral, Gemma, Phi-3
Complexity	Higher (multiple encoders)	Lower
Cost & Compute	Higher	Lower

9. Open-Source Multimodal Models

Model	Organization	Modalities	Features
Llama 3.2 Vision	Meta	T + 🖼	Open-source, text + image
Qwen-VL	Alibaba	T + 🖼	Multimodal, strong performance
InternVL	OpenGVLab	T + 🖼 + ▶	Open-source, vision + video
CogVLM	THUDM	T + 🖼	Multimodal understanding
DeepSeek-VL	DeepSeek	T + 🖼	High performance, open-source
Florence-2	Microsoft	T + 🖼	Vision tasks (caption, detection, etc.)

10. Tools & Frameworks

Tool/Framework	Purpose
Hugging Face	Models, datasets, easy deployment
Transformers	Pre-trained models (text, vision, audio)
Llama.cpp	Run LLMs locally (CPU/GPU)
vLLM	Fast inference for LLMs
Ollama	Run open-source models locally
LangChain	Build AI applications with LLMs (supports multimodal)
Gradio	Create demo web apps easily

11. Challenges

- ⚠ High computational cost (especially for large multimodal models).
- ⚠ Large data requirements (text + images + audio + video).
- ⚠ Safety and bias issues (misinformation, deepfakes).
- ⚠ Handling different modalities effectively.
- ⚠ Deployment and real-time performance.
- ⚠ Privacy and data security concerns.

12. Future Trends

- ☑ More powerful and efficient multimodal models.
- ☑ Real-time multimodal AI (video + audio + text).
- ☑ Open-source multimodal models becoming stronger.
- ☑ Better integration with robotics and real-world systems.
- ☑ Personal AI assistants with multimodal understanding.
- ☑ Increased focus on safety, alignment, and ethical AI.

13. Key Takeaways

- ☑ Multimodal AI can understand and generate multiple types of data (text, image, audio, video).
- ☑ Open-source LLMs make AI more accessible and customizable.
- ☑ Popular models: Llama, Mistral, Gemma, Phi, Qwen, DeepSeek.
- ☑ Frameworks like Hugging Face and LangChain help build multimodal AI applications.
- ☑ The future is more powerful, efficient, and real-world multimodal AI systems.

Multiple Modalities + Open Models + Real-World Impact = More Capable & Accessible AI

@codedsoul_05

19. Build a Real-World Gen AI Project

1. Project Overview

- We will build a real-world AI Chat Assistant with document Q&A using Open Source LLMs.
- Features:
  - ☑ Chat with your documents (PDF, DOC, TXT)
  - ☑ Get accurate answers with sources
  - ☑ Modern web interface (React)
  - ☑ Uses open-source models (LLM)
  - ☑ Deployable on your own server or cloud

Real-World Use Cases:

- Company knowledge base
- Insurance/product policy assistant
- HR & internal documentation
- Customer support bot

2. Tech Stack

Frontend	Backend
React + Vite Tailwind CSS	Python, FastAPI Uvicorn

LLM (Open Source)

Llama 3.1/3.2 (via Ollama)
Mistral (via Ollama)

Embeddings

BAAI/bge-small-en
(or nomic-embed-text)

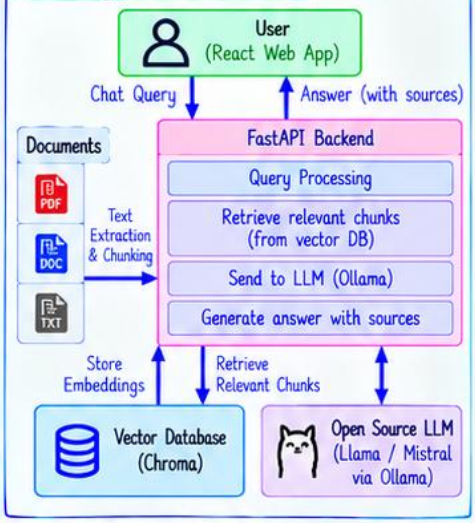
Vector Database

Chroma (easy & open-source)

Deployment

Docker (optional),
Render/Local

3. Project Architecture



4. Folder Structure

```
genai-doc-assistant/
├── backend/
│   ├── main.py
│   ├── rag.py
│   ├── utils.py
│   └── requirements.txt
└── frontend/
    ├── src/
    │   ├── components/
    │   ├── App.jsx
    │   ├── main.jsx
    │   └── index.css
    ├── data/
    │   └── uploads/
    ├── README.md
    └── docker-compose.yml
```

5. Setup Environment

1. Install Python packages

```
pip install fastapi uvicorn
langchain chromadb pypdf
sentence-transformers
python-multipart
```

2. Install Ollama (LLM)

```
# Download from https://ollama.com
ollama pull llama3.2
# or
ollama pull mistral
```

3. Create project folders

```
mkdir genai-doc-assistant
cd genai-doc-assistant
mkdir backend frontend data
```

6. Backend Code (FastAPI)

backend/main.py

```python
from fastapi import FastAPI, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
from rag import process_document, ask_question

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    path = f"data/{file.filename}"
    with open(path, "wb") as f:
        f.write(await file.read())
    process_document(path)
    return {"message": "File uploaded and processed"}

@app.post("/chat")
async def chat(query: dict):
    answer = ask_question(query["question"])
    return {"answer": answer}
```

7. RAG Logic (rag.py)

```python
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma
from langchain.llms import Ollama
from langchain.chains import RetrievalQA

embeddings = HuggingFaceEmbeddings(
    model_name="BAAI/bge-small-en"
)
vectordb = Chroma(persist_directory="./chroma_db",
                  embedding_function=embeddings)
llm = Ollama(model="llama3.2")
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectordb.as_retriever(),
    return_source_documents=True

def process_document(file_path):
    loader = PyPDFLoader(file_path)
    docs = loader.load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000, chunk_overlap=200)
    chunks = splitter.split_documents(docs)
    vectordb.add_documents(chunks)

def ask_question(question):
    result = qa_chain({"query": question})
    return {
        "answer": result["result"],
        "sources": [doc.metadata.get("source")
            for doc in result["source_documents"]]
    }
```

8. Frontend (React)

frontend/src/App.jsx

```jsx
import { useState } from "react"

function App() {
  const [file, setFile] = useState(null)
  const [question, setQuestion] = useState("")
  const [answer, setAnswer] = useState("")

  const uploadFile = async () => {
    const formData = new FormData()
    formData.append("file", file)
    await fetch("http://localhost:8000/upload", {
      method: "POST",
      body: formData
    })
    alert("File uploaded!")
  }

  const askQuestion = async () => {
    const res = await fetch("http://localhost:8000/chat",{
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ question })
    const data = await res.json()
    setAnswer(data.answer)

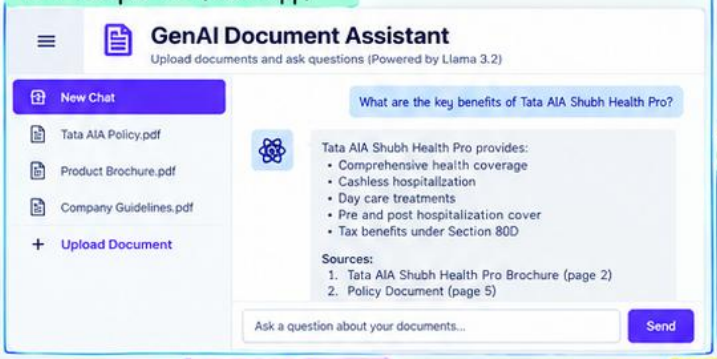
  return ( /* UI for upload, input and show answer */)
}
export default App
```

Use Tailwind CSS for a modern UI
(with chat interface, file upload, and sources).

9. Run the Project

1. Start Ollama
   ollama serve
2. Run FastAPI backend
   uvicorn main:app --reload
   (http://localhost:8000)
3. Run React frontend
   cd frontend
   npm install
   npm run dev
   (http://localhost:5173)

10. Sample UI (Web App)



11. Deployment Options

- Local – Run on your laptop (Ollama + FastAPI + React)
- Docker – Use docker-compose
- Cloud – Render / AWS / GCP / Azure (deploy backend and frontend)

12. Enhancements (Next Steps)

- ☑ Support multiple document formats (DOCX, CSV)
- ☑ Show page numbers and highlighted text
- ☑ Chat history and multiple conversations
- ☑ User authentication (login)
- ☑ Deploy with Docker
- ☑ Use better models (Llama 3.3, Mistral)
- ☑ Add voice input/output (multimodal)
- ☑ Add charts/tables understanding
- ☑ Build domain-specific assistants (e.g., insurance)

13. Real-World Use Cases

Insurance Product Assistant | HR Policy Bot | Customer Support | Research Assistant | Educational Tutor | Legal/Finance Document Q&A

14. Key Takeaways

- ☑ You built a working Gen AI application using open-source tools.
- ☑ You implemented RAG (Retrieval Augmented Generation).
- ☑ You can extend it for real business use cases.
- ☑ No API costs – runs on your own models (Ollama).
- ☑ Easily deployable and customizable.

15. Useful Resources

- Ollama: https://ollama.com
- LangChain: https://www.langchain.com
- Chroma: https://www.trychroma.com
- Llama 3 (Meta): https://llama.meta.com
- FastAPI: https://fastapi.tiangolo.com
- React: https://react.dev

Upload Documents → Retrieve Knowledge → Ask Questions → Get Accurate Answers with Sources

@codedsoul_05

20. Generative AI Cheat Sheet + 100 Interview Questions

Complete Revision Guide for Beginners to Advanced

1. Gen AI Quick Concepts

- **Gen AI:** Creates new content (text, image, code, audio, video).
- **LLM:** Large Language Model (e.g., GPT, Llama).
- **Prompt:** Input given to the model.
- **Tokens:** Small pieces of text.
- **Embeddings:** Convert text to vectors.
- **RAG:** Retrieve external knowledge.
- **Fine-Tuning:** Train on your own data.
- **LoRA/PEFT:** Efficient fine-tuning.
- **Agents:** AI that can plan and use tools.
- **Multimodal:** Works with text, image, audio, video.

2. Popular LLMs

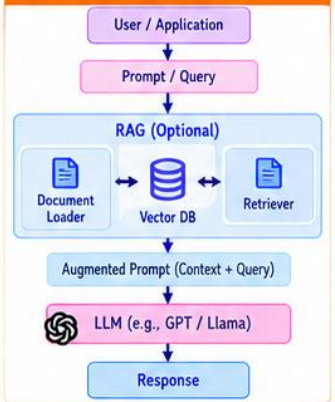
OpenAI	GPT-4o, GPT-4o mini
Meta	Llama 3.1 / 3.2
Google	Gemini 1.5 / 2.0
Anthropic	Claude 3 / 3.5
Mistral AI	Mistral Large/Medium
Microsoft	Phi-3
Alibaba	Qwen
DeepSeek	DeepSeek-VL

All are open-source or have accessible versions.

3. Model Types

Text LLMs	GPT, Llama, Mistral
Vision Models	Llama 3.2 Vision, GPT-4o
Audio Models	Whisper, SpeechT5
Multimodal	GPT-4o, Gemini, Qwen-VL
Embedding Models	text-embedding-3, BGE
Rerankers	bge-reranker, Cohere Rerank

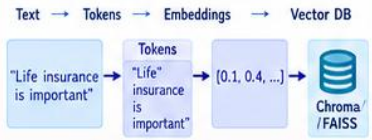
4. Gen AI Architecture



5. Key Tools & Frameworks

LangChain	Build LLM apps, tool calling
LangGraph	Agentic workflows
LlamaIndex	RAG with data
Hugging Face	Models, datasets, training
Ollama	Run open-source models locally
vLLM	Fast inference
Chroma / FAISS	Vector databases
Gradio	Build demo UI
FastAPI	Build backend APIs
Streamlit	Build web apps
Docker	Deployment
OpenAI / Anthropic APIs	Commercial LLM APIs

6. Tokenization & Embeddings



- **Tokens:** Small text units (e.g., 1–4 characters).
- **Embeddings:** Numerical vector representation.
- Used for search, similarity, RAG, clustering.

7. RAG Pipeline (Steps)

1. Load documents (PDF, DOC, Website)
2. Chunk text (split into pieces)
3. Create embeddings
4. Store in vector database (Chroma/FAISS)
5. Retrieve relevant chunks for a query
6. Send (context + query) to LLM
7. Get accurate, source-based answer

8. Fine-Tuning, LoRA & PEFT

Method	What it Does	Use Case
Full Fine-Tuning	Update all parameters	High compute, high data
PEFT (General)	Train small subset	Efficient training
LoRA	Add low-rank matrices	Popular, efficient
QLoRA	LoRA + 4-bit quantization	Very low memory
Adapters	Train small adapter layers	Modular, reusable
Prompt Tuning	Train prompt tokens	Lightweight

9. AI Agents & Tool Calling

- Agents can plan, use tools and take actions.
- Uses LLM + Tools (APIs, DB, code, web).
- Frameworks: LangChain, LangGraph.
- **Key Components:**
  - Planner (decide next step)
  - Tools (search, DB, code, API)
  - Memory (store conversation)
  - Executor (call tools)
- **Example Tasks:**
  - Search web
  - Read company documents
  - Book a meeting
  - Generate reports

10. Multimodal AI

- Understands and generates multiple types of data.
- Examples: GPT-4o, Gemini, Llama 3.2 Vision.
- Input: text, images, audio, video.
- Use Cases: image Q&A, document understanding, video analysis, speech-to-text.
- Real World: healthcare, education, e-commerce, security.

11. Real-World Project Flow



Use Cases:

- Company knowledge assistant
- Insurance product Q&A bot
- HR policy chatbot
- PDF/Document summarizer
- Customer support agent

12. Evaluation Metrics

Metric	What it Measures
Perplexity (PPL)	Text prediction quality (lower is better)
Accuracy	Correctness of answers
BLEU / ROUGE	Text generation similarity
Human Evaluation	Real-world usefulness
Faithfulness	Uses correct context
Relevance	Answer matches query
Toxicity / Bias	Safety and ethical behavior

13. Best Practices

- Use clear and specific prompts.
- Provide relevant context (RAG).
- Use open-source models to reduce cost.
- Keep data clean and updated.
- Use chunking and proper embeddings.
- Monitor and evaluate model outputs.
- Handle errors and edge cases.
- Use safety measures and moderation.
- Make the system scalable and secure.
- Log and track usage.

14. Common Challenges

- Incorrect or hallucinated answers.
- Handling large documents.
- High computational cost.
- Slow inference.
- Security and privacy risks.
- Bias and harmful content.
- Tool/API failures.
- Maintaining context in long chats.
- Evaluation is complex.
- Deployment and scaling.

15. Key Takeaways

- Gen AI can create text, image, audio, video.
- RAG improves accuracy with real data.
- Open-source LLMs are powerful and accessible.
- LoRA/PEFT makes fine-tuning affordable.
- Agents can use tools and automate tasks.
- Multimodal AI handles multiple data types.
- Use frameworks like LangChain/LangGraph.
- Real-world projects solve business problems.
- Evaluate, monitor and ensure safety.
- Gen AI is rapidly evolving with huge potential.

16. 100 Gen AI Interview Questions (Grouped)

A. Basic Concepts (1–10)

1. What is Generative AI?
2. What is an LLM?
3. What is a token?
4. What is tokonizer?
5. What is embedding?
6. What is RAG?
7. What is fine-tuning?
8. What is LoRA?
9. What is a multimodal model?
10. What are AI Agents?

B. LLMs & Model Architecture (11–20)

11. How do LLMs work?
12. What is transformer?
13. What is self-attention?
14. What is positional encoding?
15. What is context window?
16. What is temperature!?
17. What is top-k and top-p?
18. What is beam search?
19. What are model parameters?
20. How to reduce hallucinations?

C. Prompting & Output (21–30)

21. What is a prompt?
22. What are prompt engineering techniques?
23. What is zero-shot prompting?
24. What is few-shot prompting?
25. What is chain-of-thought?
26. How to design effective prompts?
27. What is system prompt?
28. What is format control?
29. How to handle long context?
30. How to get structured output (JSON)?

D. RAG & Vector DB (31–40)

31. What is RAG?
32. How does RAG work?
33. What is chunking?
34. What is embedding model?
35. What is vector database?
36. Chroma vs FAISS?
37. What is similarity search?
38. How to improve RAG accuracy?
39. What is a reranker?
40. How to handle large documents?

E. Fine-Tuning & PEFT (41–50)

41. What is fine-tuning?
42. What is PEFT?
43. What is LoRA?
44. What is QLoRA?
45. What are adapters?
46. What is prompt tuning?
47. When to use each method?
48. What data is needed?
49. What are the hardware requirements?
50. How ts evaluate a fine-tuned model?

F. LangChain & LangGraph (51–60)

51. What is LangChain?
52. What is n chains in LangChain?
53. What is LangGraph?
54. Difference between LangChain and LangGraph?
55. What is a tool in LangChain?
56. How to create a custom tool?
57. What is memory in LangChain?
58. What to build an agent?
59. What is a multi-agent system?
60. How to debug LangChain apps?

G. Agents & Tool Calling (61–70)

61. What is an AI agent?
62. How does tool calling work?
63. What are common tools?
64. How to create a custom tool?
65. What is ReAct agent?
66. What is function calling?
67. Hhw to give real-time data to agent?
68. What is multi-agent collaboration?
69. What are agent evaluation metrics?
70. What are safety concerns in agents?

H. Multimodal AI (71–80)

71. What is multimodal AI?
72. Which models support multimodal?
73. How does image understanding work?
74. How to process audio?
75. How to analyze video?
76. What is vision-language model?
77. What are real-world use cases?
78. Challenges in multimodal AI?
79. How to fine-tune multimodal models?
80. Future of multimodal AI?

I. Deployment & MLOps (91–90)

81. How to deploy LLM locally?
82. How to deploy with Docker?
83. What is vLLM?
84. How to build a scalable API with FastAPI?
85. How to deploy on cloud (AWS/GCP)?
86. What is model monitoring?
87. How to log and track usage?
88. How to handle cost optimization?
89. What are security best practices?
90. How to handle high traffic?

J. Advanced & Real-World (91–100)

91. How to build a real-world Gen AI project?
92. How to integrate with a database?
93. How to build a document Q&A system?
94. How to evaluate RAG outputs?
95. How to ensure responsible AI?
96. How to handle bias and toxic content?
97. How to build an enterprise chatbot?
98. How to add authentication and permissions?
99. How to measure business impact?
100. What are future trends in Gen AI?